

Independent sets of the three-row grid strip with
the middle cell of every second column deleted: a
sixth-order recurrence
(Lean 4 machine-checked claim of record)

Aurora0134*

September 30, 2026

Abstract

Let $z(n)$ be the number of independent sets (empty set included) of the three-row by n -column grid strip $P_3 \square P_n$ with the middle-row vertex of every odd-numbered column (0-based columns $1, 3, 5, \dots$) deleted. The AI4Math pipeline formalised this counting sequence in Lean 4 and machine-proved its sixth-order recurrence: for every $n \geq 6$, $z(n) + 15z(n-4) = 12z(n-2) + 2z(n-6)$, so each term is an integral linear combination of the terms of the same parity two and four steps back. The proof compiles with no `sorry`, and `#print axioms` reports exactly `{propext, Quot.sound, Classical.choice}`. The pipeline’s exclusivity review found no occupying record for the sequence $z(n)$ itself or for the defective-strip counting object: the even-indexed subsequence of z coincides term-by-term with the registered sequence A122011 (a 3×3 matrix-power example with no graph-theoretic content, a boundary that the recurrence signature is already indexed under), while $z(n)$ itself, its odd-indexed subsequence, and the defective-strip object are unregistered. The claim is strictly *new sequence (lower tier)*, never new mathematics and never a new recurrence. This note is a priority claim of record, not a full paper.

1 Introduction

One theorem about one integer sequence is reported here. The sequence counts independent sets of a grid strip with a periodic column defect: in

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

the three-row strip every odd column loses its middle vertex, so a column is either a full 3-vertex path or two disconnected corner vertices. The first values are $z(0) = 1, z(1) = 5, z(2) = 13, z(3) = 47, z(4) = 141, z(5) = 491, z(6) = 1499, z(7) = 5197$ (computed), and the theorem proved here is the recurrence $z(n) + 15z(n-4) = 12z(n-2) + 2z(n-6)$ for all $n \geq 6$. The proof was produced by the AI4Math pipeline, in which LLM workers generate statements and proof scripts and the Lean 4 kernel is the sole adjudicator at every gate: the statement is frozen by hash before proving, and a separate audit re-computes the symmetric difference, scans the whole file for `sorry/admit` including comments, and re-prints the axiom dependencies [1]. This note is a priority claim of record deposited so that a dated, citable public record exists ahead of the narrative paper; no literature review is attempted here.

2 Statement

The formalisation works directly with the vertex set `Fin 3 × Fin n`: `holes` is the set of middle-row (row 1) vertices in odd-numbered columns, and `indepCountHoles` counts vertex subsets that are disjoint from the hole set and independent in the box product `pathGraph 3 □ pathGraph n`; the sequence is `z n := indepCountHoles 3 n (holes 3 n)`. The recurrence is stated with natural-number truncated subtraction, self-consistent under the hypothesis $6 \leq n$.

Theorem 1 (sixth-order recurrence). *For every $n : \mathbb{N}$ with $6 \leq n$,*

$$zn + 15 \cdot z(n-4) = 12 \cdot z(n-2) + 2 \cdot z(n-6).$$

Listing 1: Frozen base definitions and the head of Theorem 1, verbatim lines 21–48 of `formalized/statement.lean` (sha256 b726f0d8): the two decidability instances, `indepCountHoles`, `holes`, `z`, and the theorem statement. The frozen file’s proof body and its `native_decide` data-check lines are excluded.

```

1  -- 底座三件（照抄 smoke/B-deep-smokes.lean 已实测通过的写法）——
2
3  instance (n : ℕ) : DecidableRel (pathGraph n).Adj := fun _ _ =>
4    decidable_of_iff _ pathGraph_adj.symm
5
6  instance {α β : Type*} [DecidableEq α] [DecidableEq β] (G :
   SimpleGraph α) (H : SimpleGraph β)

```

```

7      [DecidableRel G.Adj] [DecidableRel H.Adj] : DecidableRel (G
8      □ H).Adj := fun _ _ =>
9      decidable_of_iff _ boxProd_adj.symm
10
11 /-- 与挖点集 `holes` 不交的独立集数 = 挖去 `holes` 后剩余图的独立集
12 数。 -/
13 def indepCountHoles (m n : ℕ) (holes : Finset (Fin m × Fin n)) :
14   ℕ :=
15   ((univ : Finset (Finset (Fin m × Fin n))).filter (fun s =>
16     Disjoint s holes ∧ (pathGraph m □ pathGraph n).IsIndepSet s))
17   .card
18
19 -- 缺陷集与计数列  $z$  ——
20
21 /-- 隔列挖中点缺陷集: `Fin m × Fin n` 中行号 = 1 (中行) 且列号为奇数
22 的点,
23 0 基行列, 即  $(1, 1), (1, 3), (1, 5), \dots$  (`Finset.univ.filter` □
24 径, 不经 `Subgraph.induce`)。 -/
25 def holes (m n : ℕ) : Finset (Fin m × Fin n) :=
26   (univ : Finset (Fin m × Fin n)).filter (fun p => p.1.val = 1 ∧
27     p.2.val % 2 = 1)
28
29 /-- 三行带每隔一列挖去中行顶点所得图的独立集数列。 -/
30 def z (n : ℕ) : ℕ := indepCountHoles 3 n (holes 3 n)
31
32 -- 主定理 (proof 占位: 本任务只产 statement) ——
33
34 /--  $z$  的六阶递推 ( $\mathbb{N}$  截断减法形, 系数  $15/12/2$  与方向依任务书不动)。
35 -/
36 theorem c_z_rec :  $\forall n : \mathbb{N}, 6 \leq n \rightarrow z\ n + 15 * z\ (n - 4) = 12 * z\ (n - 2) + 2 * z\ (n - 6)$  := by

```

Listing 2: Frozen attack interface and matrix certificates, verbatim lines 18–163 of formalized/interface.lean (sha256 66c96c3e): the frozen statement segment, the profile/transfer definitions P5/Q4/allowed/ValidProf/validCount, the matrices Amat/Bmat/Tmat and the two decide-closed certificates cert_ones/cert_vec0.

```

1  -- =====
2  -- 段一: 冻结 statement (逐字节, 禁改)
3  -- =====
4
5  instance (n : ℕ) : DecidableRel (pathGraph n).Adj := fun _ _ =>
6    decidable_of_iff _ pathGraph_adj.symm

```

```

7
8 instance {α β : Type*} [DecidableEq α] [DecidableEq β] (G :
  SimpleGraph α) (H : SimpleGraph β)
9   [DecidableRel G.Adj] [DecidableRel H.Adj] : DecidableRel (G
    □ H).Adj := fun _ _ =>
10   decidable_of_iff _ boxProd_adj.symm
11
12 /-- 与挖点集 `holes` 不交的独立集数 = 挖去 `holes` 后剩余图的独立集
    数。 -/
13 def indepCountHoles (m n : ℕ) (holes : Finset (Fin m × Fin n)) :
    ℕ :=
14   ((univ : Finset (Finset (Fin m × Fin n))).filter (fun s =>
15     Disjoint s holes ∧ (pathGraph m □ pathGraph n).IsIndepSet s))
    .card
16
17 /-- 隔列挖中点缺陷集: `Fin m × Fin n` 中行号 = 1 (中行) 且列号为奇数
    的点,
18 0 基行列, 即 (1, 1), (1, 3), (1, 5), ... (`Finset.univ.filter` □
    径, 不经 `Subgraph.induce`)。 -/
19 def holes (m n : ℕ) : Finset (Fin m × Fin n) :=
20   (univ : Finset (Fin m × Fin n)).filter (fun p => p.1.val = 1 ∧
    p.2.val % 2 = 1)
21
22 /-- 三行带每隔一列挖去中行顶点所得图的独立集数列。 -/
23 def z (n : ℕ) : ℕ := indepCountHoles 3 n (holes 3 n)
24
25 -- =====
26 -- 段二: 攻证接口 (新增; 证明员只读引用, 不改写)
27 -- =====
28
29 /-- 列切片: 点集 `s` 在第 `j` 列上的行集合。 -/
30 def colSlice (n : ℕ) (j : Fin n) (s : Finset (Fin 3 × Fin n)) :
    Finset (Fin 3) :=
31   univ.filter (fun r => (r, j) ∈ s)
32
33 /-- 偶列 (无洞) 允许的剖面集: 全 5 个竖直独立剖面。 -/
34 def Pprof : Finset (Finset (Fin 3)) := {∅, {0}, {1}, {2}, {0, 2}}
35
36 /-- 奇列 (有洞, 禁中行) 允许的剖面集: 4 个不含行 1 的竖直独立剖面。
    -/
37 def Qprof : Finset (Finset (Fin 3)) := {∅, {0}, {2}, {0, 2}}
38
39 /-- 第 `j` 列 (0 基) 允许的剖面集: 奇列禁中行, 偶列全允许。 -/
40 def allowed (j : ℕ) : Finset (Finset (Fin 3)) := if j % 2 = 1 then
    Qprof else Pprof

```

```

41
42 /-- 剖面函数合法性：每列剖面合法，且相邻两列剖面不交。 -/
43 def ValidProf (n : ℕ) (f : Fin n → Finset (Fin 3)) : Prop :=
44   (∀ j : Fin n, f j ∈ allowed j.val) ∧
45   (∀ j j' : Fin n, (pathGraph n).Adj j j' → Disjoint (f j) (f j
46     '))
47 /-- `ValidProf` 的可判定性实例（显式给出，使 `validCount` 的 `
48   filter` 实例唯一、
49   定义上透明：`validCount n = (univ.filter (ValidProf n)).card` 按 `
50   rfl` 成立）。
51   不显式给会走 `classical` 盲选，证明里另写的同形 `filter` 会拿到不同实
52   例，两侧
53   命题相等但不同定义，`card_bij` 直接卡死。 -/
54
55 instance (n : ℕ) : DecidablePred (ValidProf n) := fun f => by
56   unfold ValidProf
57   infer_instance
58
59 /-- 合法剖面函数计数。
60   (`noncomputable`：上面的可判定实例内部含经典判定；公理走白名单内 `
61   Classical.choice`。) -/
62 noncomputable def validCount (n : ℕ) : ℕ :=
63   ((univ : Finset (Fin n → Finset (Fin 3))).filter (ValidProf n))
64   .card
65
66 /-- 由剖面函数重建点集：第 `j` 列取 `f j` 中的行。 -/
67 def reconstruct (n : ℕ) (f : Fin n → Finset (Fin 3)) : Finset (
68   Fin 3 × Fin n) :=
69   univ.biUnion (fun j => (f j).image (fun r => (r, j)))
70
71 -- =====
72 -- 段三：转移矩阵与证书（`decide` 一击闭；已严格模式实测 `exit 0`）
73 -- =====
74
75 /-- 剖面枚举（偶列侧，5 个）。 -/
76 def P5 : Fin 5 → Finset (Fin 3)
77 | 0 => ∅
78 | 1 => {0}
79 | 2 => {1}
80 | 3 => {2}
81 | 4 => {0, 2}
82
83 /-- 剖面枚举（奇列侧，4 个）。 -/
84 def Q4 : Fin 4 → Finset (Fin 3)
85 | 0 => ∅

```

```

79 | 1 => {0}
80 | 2 => {2}
81 | 3 => {0, 2}
82
83 /-- 偶→奇转移矩阵: `Amat i j = 1` 当且仅当剖面 `P5 i` 与 `Q4 j` 不
   交。 -/
84 def Amat : Matrix (Fin 5) (Fin 4) ℤ := fun i j => if Disjoint (P5
   i) (Q4 j) then 1 else 0
85
86 /-- 奇→偶转移矩阵: `Bmat i j = 1` 当且仅当剖面 `Q4 i` 与 `P5 j` 不
   交。 -/
87 def Bmat : Matrix (Fin 4) (Fin 5) ℤ := fun i j => if Disjoint (Q4
   i) (P5 j) then 1 else 0
88
89 /-- 双步转移矩阵 `T = A·B` (5×5)。 -/
90 def Tmat : Matrix (Fin 5) (Fin 5) ℤ := Amat * Bmat
91
92 /-- 全 1 向量 (5 维)。 -/
93 def ones5 : Fin 5 → ℤ := fun _ => 1
94
95 /-- 全 1 向量 (4 维)。 -/
96 def ones4 : Fin 4 → ℤ := fun _ => 1
97
98 /-- 偶侧初值向量 `vec0 = A·1` (5 维; 分量 = 与 `P5 i` 不交的奇剖面个
   数)。 -/
99 def vec0 : Fin 5 → ℤ := Amat_v * ones4
100
101 /-- 证书一: `p(T)·1 = 0` (点态), `p(x) = x³ - 12x² + 15x - 2`。 -/
102 theorem cert_ones : ∀ i : Fin 5,
103   ((Tmat ^ 3) _v * ones5) i
104   = 12 * (((Tmat ^ 2) _v * ones5) i) - 15 * ((Tmat ^ 1) _v * ones5
   ) i + 2 * ones5 i := by
105   decide
106
107 /-- 证书二: `p(T)·vec0 = 0` (点态)。 -/
108 theorem cert_vec0 : ∀ i : Fin 5,
109   ((Tmat ^ 3) _v * vec0) i
110   = 12 * (((Tmat ^ 2) _v * vec0) i) - 15 * ((Tmat ^ 1) _v * vec0)
   i + 2 * vec0 i := by
111   decide
112
113 -- =====
114 -- 段四: 名册核验 (本文件编译即证下列名字存在; 供证据包引用)
115 -- =====
116

```

```

117 #check @Finset.card_bij
118 #check @Finset.card_biUnion
119 #check @Finset.card_sigma
120 #check @Finset.card_filter
121 #check @Finset.card_image_of_injOn
122 #check @Finset.mem_biUnion
123 #check @Finset.mem_image
124 #check @Finset.disjoint_left
125 #check @Finset.filter_filter
126 #check @Finset.sum_filter
127 #check @Finset.sum_boole
128 #check @Finset.sum_biUnion
129 #check @Finset.card_eq_sum_ones
130 #check @Fintype.card_congr
131 #check @Fintype.card_fin
132 #check @SimpleGraph.isIndepSet_iff
133 #check @SimpleGraph.boxProd_adj
134 #check @SimpleGraph.pathGraph_adj
135 #check @Nat.strong_induction_on
136 #check @Fin.last
137
138 -- 攻证期实测可用的 API 名册 (2026-09-30 探针核实) ——
139 #check @Finset.card_bij -- 依赖型签名:  $i : (a : \alpha) \rightarrow a \in s \rightarrow \beta$ 
    (带成员性证明)
140 #check @Fintype.card_congr -- 取 Equiv
141 #check @Finset.card_image_of_injOn
142 #check @Matrix.mulVec -- 记号  $\text{`}M_{\mathbf{v}} * \mathbf{v}\text{'}$ :  $(M_{\mathbf{v}} * \mathbf{v}) \ i = \sum_j \ j, \ M \ i \ j *$ 
     $\mathbf{v} \ j$ 
143 #check @Matrix.vecMul -- 记号  $\text{`}\mathbf{v}_{\mathbf{v}} * M\text{'}$ :  $(\mathbf{v}_{\mathbf{v}} * M) \ j = \sum_i \ i, \ \mathbf{v} \ i * M$ 
     $i \ j$ 
144 #check @Finset.sum_image
145 -- 实测不存在: `Matrix.dotProduct` (勿回喂该名字)

```

The complete final file is reproduced verbatim in Section 3 (whole file, no excerpting); it also ships in this deposit under `proofs/`. The frozen statement file carries the theorem body as a `sorry` placeholder (that is its role: the formalisation department freezes statements, the proving department replaces bodies), followed by five `native_decide` data-check lines for the initial values $z(0..4) = 1, 5, 13, 47, 141$; both lie outside every listing above. In the final file no `native_decide` occurs in code and the initial values are reached through the transfer-matrix route instead, so the axiom output stays on the whitelist (Section 4).

3 Proof

The proof is a three-lane assembly, all lanes machine-checked. Lane A (bridge) turns the graph-side count into a column-profile count: a vertex set is sliced into per-column row sets (`colSlice`), a profile function rebuilds a vertex set (`reconstruct`), the two maps are mutually inverse, and `Finset.card_bij` transports the count, giving $\mathbf{z} \cdot \mathbf{n} = \text{validCount} \cdot \mathbf{n}$ (`br_z_eq_validCount`), where `validCount` counts functions assigning each column a legal profile (the five vertically independent row sets on even columns, the four middle-row-free ones on odd columns) with adjacent columns disjoint. Lane B (transfer) packages the two-step recurrence of these profile counts into matrix form: filling the last column of a width- $(m+2)$ strip decomposes over the previous column's profile with weight the indicator of profile disjointness (`tr_fillEnd_step`, a `card_bij/card_biUnion` argument), and joint induction on the step count gives $\text{validCount}(2k+1) = \sum_i (T^k \mathbf{1})_i$ and $\text{validCount}(2k+2) = \sum_j (T^k \vec{v}_0)_j$ with $T = \text{Tmat} = \text{Amat} \cdot \text{Bmat}$, $\vec{v}_0 = \text{vec0} = \text{Amat} \cdot \mathbf{1}_4$ (`tr_validCount_odd`, `tr_validCount_even`, `tr_validCount_zero`). Lane C (algebra) uses the certificate polynomial $p(x) = x^3 - 12x^2 + 15x - 2$: the two `decide`-closed certificates state $p(T)\mathbf{1} = 0$ and $p(T)\vec{v}_0 = 0$ pointwise, which by associativity of matrix powers yields the order-3 pointwise recurrences of $w(k) = T^k \mathbf{1}$ and $u(k) = T^k \vec{v}_0$, hence of their sums (`asm_w_rec_sum`, `asm_u_rec_sum`); the main theorem splits on the parity of n (the odd lane needs no boundary case; the even lane has exactly one, $n = 6$, closed by a `decide`-ed integer identity of four matrix sums, $1499 + 15 \cdot 13 = 12 \cdot 141 + 2 \cdot 1$).

Listing 3: Complete proof file of Theorem 1, verbatim lines 1–1188 (whole file) of `attempts/final-c_z_rec.lean` (sha256 9a837835). Mixed line endings on the working disk (lines 1–485 and 953–1189 LF, 486–952 CRLF) are spliced LF-normalised per the chorded/ladder precedent; byte-exact original in `proofs/`.

```

1  import Mathlib
2
3  open Finset SimpleGraph Matrix
4
5  /-!
6  # comb-12 接口冻结件 (formalized/interface.lean, 2026-09-30)
7
8  任务: tasks/20260930-comb12-grid-indep (pool-comb-12 全流程)
9  冻结 statement 来源: tasks/20260929-preselect-deeprecheck-01/attempts/L13C
   /statement.lean
10 (sha256

```

```

b726f0d8e01672b39a93ce0c256f5036014bc80f7f0b6a5fcb484272d805c884)
11
12 本文件 = (1)冻结 statement 的声明段 (逐字节) + (2)攻证接口 (新增 def,
    kernel 实测可编)
13 + (3)转移矩阵证书 (`decide` 一击闭, 严格模式实测 exit 0)。
14
15 **接口纪律**: 本文件在最终稿中逐字节保留; 证明员只读引用, 不得改写其中任何声
    明。
16 -/
17
18 -- =====
19 -- 段一: 冻结 statement (逐字节, 禁改)
20 -- =====
21
22 instance (n : ℕ) : DecidableRel (pathGraph n).Adj := fun _ _ =>
23   decidable_of_iff _ pathGraph_adj.symm
24
25 instance {α β : Type*} [DecidableEq α] [DecidableEq β] (G : SimpleGraph
    α) (H : SimpleGraph β)
26   [DecidableRel G.Adj] [DecidableRel H.Adj] : DecidableRel (G □ H).Adj
    := fun _ _ =>
27     decidable_of_iff _ boxProd_adj.symm
28
29 /-- 与挖点集 `holes` 不交的独立集数 = 挖去 `holes` 后剩余图的独立集数。 -/
30 def indepCountHoles (m n : ℕ) (holes : Finset (Fin m × Fin n)) : ℕ :=
31   ((univ : Finset (Finset (Fin m × Fin n))).filter (fun s =>
32     Disjoint s holes ∧ (pathGraph m □ pathGraph n).IsIndepSet s)).card
33
34 /-- 隔列挖中点缺陷集: `Fin m × Fin n` 中行号 = 1 (中行) 且列号为奇数的点,
    0 基行列, 即 (1, 1), (1, 3), (1, 5), ... (`Finset.univ.filter` 口径, 不经 `
    Subgraph.induce`)。 -/
35
36 def holes (m n : ℕ) : Finset (Fin m × Fin n) :=
37   (univ : Finset (Fin m × Fin n)).filter (fun p => p.1.val = 1 ∧ p.2.val
    % 2 = 1)
38
39 /-- 三行带每隔一列挖去中行顶点所得图的独立集数列。 -/
40 def z (n : ℕ) : ℕ := indepCountHoles 3 n (holes 3 n)
41
42 -- =====
43 -- 段二: 攻证接口 (新增; 证明员只读引用, 不改写)
44 -- =====
45
46 /-- 列切片: 点集 `s` 在第 `j` 列上的行集合。 -/
47 def colSlice (n : ℕ) (j : Fin n) (s : Finset (Fin 3 × Fin n)) : Finset (
    Fin 3) :=
48   univ.filter (fun r => (r, j) ∈ s)
49
50 /-- 偶列 (无洞) 允许的剖面集: 全 5 个竖直独立剖面。 -/
51 def Pprof : Finset (Finset (Fin 3)) := {∅, {0}, {1}, {2}, {0, 2}}

```

```

52
53 /-- 奇列（有洞，禁中行）允许的剖面集：4 个不含行 1 的竖直独立剖面。 -/
54 def Qprof : Finset (Finset (Fin 3)) := {∅, {0}, {2}, {0, 2}}
55
56 /-- 第 `j` 列（0 基）允许的剖面集：奇列禁中行，偶列全允许。 -/
57 def allowed (j : ℕ) : Finset (Finset (Fin 3)) := if j % 2 = 1 then Qprof
58   else Pprof
59
60 /-- 剖面函数合法性：每列剖面合法，且相邻两列剖面不交。 -/
61 def ValidProf (n : ℕ) (f : Fin n → Finset (Fin 3)) : Prop :=
62   (∀ j : Fin n, f j ∈ allowed j.val) ∧
63   (∀ j j' : Fin n, (pathGraph n).Adj j j' → Disjoint (f j) (f j'))
64
65 /-- `ValidProf` 的可判定性实例（显式给出，使 `validCount` 的 `filter` 实例
66   唯一、
67   定义上透明：`validCount n = (univ.filter (ValidProf n)).card` 按 `rfl` 成
68   立）。
69   不显式给会走 `classical` 盲选，证明里另写的同形 `filter` 会拿到不同实例，两侧
70   命题相等但不同定义，`card_bij` 直接卡死。 -/
71 instance (n : ℕ) : DecidablePred (ValidProf n) := fun f => by
72   unfold ValidProf
73   infer_instance
74
75 /-- 合法剖面函数计数。
76   (`noncomputable`：上面的可判定实例内部含经典判定；公理走白名单内 `Classical
77   .choice`。) -/
78 noncomputable def validCount (n : ℕ) : ℕ :=
79   ((univ : Finset (Fin n → Finset (Fin 3))).filter (ValidProf n)).card
80
81 /-- 由剖面函数重建点集：第 `j` 列取 `f j` 中的行。 -/
82 def reconstruct (n : ℕ) (f : Fin n → Finset (Fin 3)) : Finset (Fin 3 ×
83   Fin n) :=
84   univ.biUnion (fun j => (f j).image (fun r => (r, j)))
85
86 -- =====
87 -- 段三：转移矩阵与证书（`decide` 一击闭；已严格模式实测 `exit 0`）
88 -- =====
89
90 /-- 剖面枚举（偶列侧，5 个）。 -/
91 def P5 : Fin 5 → Finset (Fin 3)
92 | 0 => ∅
93 | 1 => {0}
94 | 2 => {1}
95 | 3 => {2}
96 | 4 => {0, 2}
97
98 /-- 剖面枚举（奇列侧，4 个）。 -/
99 def Q4 : Fin 4 → Finset (Fin 3)
100 | 0 => ∅

```

```

96 | 1 => {0}
97 | 2 => {2}
98 | 3 => {0, 2}
99
100 /-- 偶→奇转移矩阵: `Amat i j = 1` 当且仅当剖面 `P5 i` 与 `Q4 j` 不交。 -/
101 def Amat : Matrix (Fin 5) (Fin 4) ℤ := fun i j => if Disjoint (P5 i) (Q4
    j) then 1 else 0
102
103 /-- 奇→偶转移矩阵: `Bmat i j = 1` 当且仅当剖面 `Q4 i` 与 `P5 j` 不交。 -/
104 def Bmat : Matrix (Fin 4) (Fin 5) ℤ := fun i j => if Disjoint (Q4 i) (P5
    j) then 1 else 0
105
106 /-- 双步转移矩阵 `T = A·B` (5×5)。 -/
107 def Tmat : Matrix (Fin 5) (Fin 5) ℤ := Amat * Bmat
108
109 /-- 全 1 向量 (5 维)。 -/
110 def ones5 : Fin 5 → ℤ := fun _ => 1
111
112 /-- 全 1 向量 (4 维)。 -/
113 def ones4 : Fin 4 → ℤ := fun _ => 1
114
115 /-- 偶侧初值向量 `vec0 = A·1` (5 维; 分量 = 与 `P5 i` 不交的奇剖面个数)。
    -/
116 def vec0 : Fin 5 → ℤ := Amat * ones4
117
118 /-- 证书一: `p(T)·1 = 0` (点态),  $p(x) = x^3 - 12x^2 + 15x - 2$ 。 -/
119 theorem cert_ones : ∀ i : Fin 5,
120   ((Tmat ^ 3) * ones5) i
121   = 12 * ((Tmat ^ 2) * ones5) i - 15 * ((Tmat ^ 1) * ones5) i + 2
    * ones5 i := by
122   decide
123
124 /-- 证书二: `p(T)·vec0 = 0` (点态)。 -/
125 theorem cert_vec0 : ∀ i : Fin 5,
126   ((Tmat ^ 3) * vec0) i
127   = 12 * ((Tmat ^ 2) * vec0) i - 15 * ((Tmat ^ 1) * vec0) i + 2 *
    vec0 i := by
128   decide
129
130 -- =====
131 -- 段四: 名册核验 (本文件编译即证下列名字存在; 供证据包引用)
132 -- =====
133
134 #check @Finset.card_bij
135 #check @Finset.card_biUnion
136 #check @Finset.card_sigma
137 #check @Finset.card_filter
138 #check @Finset.card_image_of_injOn
139 #check @Finset.mem_biUnion

```

```

140 #check @Finset.mem_image
141 #check @Finset.disjoint_left
142 #check @Finset.filter_filter
143 #check @Finset.sum_filter
144 #check @Finset.sum_boole
145 #check @Finset.sum_biUnion
146 #check @Finset.card_eq_sum_ones
147 #check @Fintype.card_congr
148 #check @Fintype.card_fin
149 #check @SimpleGraph.isIndepSet_iff
150 #check @SimpleGraph.boxProd_adj
151 #check @SimpleGraph.pathGraph_adj
152 #check @Nat.strong_induction_on
153 #check @Fin.last
154
155 -- 攻证期实测可用的 API 名册 (2026-09-30 探针核实) ——
156 #check @Finset.card_bij -- 依赖型签名:  $i : (a : \alpha) \rightarrow a \in s \rightarrow \beta$  (带成员性证明)
157 #check @Fintype.card_congr -- 取 Equiv
158 #check @Finset.card_image_of_injOn
159 #check @Matrix.mulVec -- 记号  $M \cdot v$ :  $(M \cdot v)_i = \sum_j M_{ij} \cdot v_j$ 
160 #check @Matrix.vecMul -- 记号  $v \cdot M$ :  $(v \cdot M)_j = \sum_i v_i \cdot M_{ij}$ 
161 #check @Finset.sum_image
162 -- 实测不存在: Matrix.dotProduct (勿回喂该名字)
163
164 /-!
165 ## 攻证须知 (2026-09-30 实测, 写 Lean 前必读)
166
167 1. **`decide` 无法从图定义直接算 z: example : z 0 = 1 := by decide` 实测失败
168    (reduction 在 `instDecidableEqNat (z 0) 1` 处卡死)。 z 的一切具体取值都必须走
169    `z = validCount = 矩阵表达式` 路线后用 `decide` 闭 (5×5 整数矩阵规模,
170    `cert_ones`/`cert_vec0` 已实测 `decide` 一击闭)。
171    **禁止用 `native_decide`: 它引入 `Lean.ofReduceBool`, 不在公理白名单
172    {propext, Quot.sound, Classical.choice} 内, 入库即拒收 (宪条 3)。
173 2. `validCount n = (univ.filter (ValidProf n)).card` 按 `rfl` 成立 (见上方显式实例);
174    证明里可直接 `unfold validCount` 或 `show`。
175 3. `z 0 = 1` 不要从图定义硬算: 走 `z 0 = validCount 0` (桥梁引理) +
176    `validCount 0 = 1` (`Fin 0 → Finset (Fin 3)` 是 subsingleton, univ 恰一元素且合法)。
177 -/
178 -- =====
179 -- 段 A · 桥梁恒等式 `z n = validCount n` (攻证员: 桥梁证明员)
180 -- 前缀纪律: 全部新引理 `br_` 开头 (guide §7.2), 不含接口已有声明。
181 -- 数学路线: guide §3 (Finset 双射: 独立集 ↔ 合法剖面函数)
182 -- =====
183

```

```

184 /-!
185 ### A0. 两侧计数对象与定义展开
186
187 `br_S1` / `br_S2` 与 `indepCountHoles` / `validCount` 的 filter 同形,
188 `DecidablePred` 实例链与 interface 段一/段二完全一致 (`card_bij` 实例陷阱的
189 防法: 不另写同形 filter, 只复用这两个 def)。
190 -/
191
192 /-- 桥侧辅助: `z n` 所数的对象 = 三行  $n$  列图中与 `holes 3 n` 不交的独立集。
193 -/
194 def br_S1 (n : ℕ) : Finset (Finset (Fin 3 × Fin n)) :=
195   (univ : Finset (Finset (Fin 3 × Fin n))).filter (fun s =>
196     Disjoint s (holes 3 n) ∧ (pathGraph 3 ⊆ pathGraph n).IsIndepSet s)
197
198 /-- 桥侧辅助: `validCount n` 所数的对象 = 合法剖面函数。 -/
199 def br_S2 (n : ℕ) : Finset (Fin n → Finset (Fin 3)) :=
200   (univ : Finset (Fin n → Finset (Fin 3))).filter (ValidProf n)
201
202 theorem br_z_eq_card_S1 (n : ℕ) : z n = (br_S1 n).card := rfl
203
204 theorem br_validCount_eq_card_S2 (n : ℕ) : validCount n = (br_S2 n).card
205   := rfl
206
207 theorem br_mem_S1 (n : ℕ) (s : Finset (Fin 3 × Fin n)) (hd : Disjoint s
208   (holes 3 n))
209   (hi : (pathGraph 3 ⊆ pathGraph n).IsIndepSet s) : s ∈ br_S1 n :=
210   Finset.mem_filter.mpr ⟨Finset.mem_univ s, hd, hi⟩
211
212 theorem br_mem_S2 (n : ℕ) (f : Fin n → Finset (Fin 3)) (h : ValidProf n
213   f) :
214   f ∈ br_S2 n := Finset.mem_filter.mpr ⟨Finset.mem_univ f, h⟩
215
216 /-!
217 ### A1. 成员性小工具 (列切片 / 重建 / 缺陷集 / 邻接 / 独立集改写)
218
219 全部按 `defeq` 释义展开 (`have h' : <展开形> := h`) , 不依赖 `rw` 对
220 beta-可约元的匹配, 规避 guide §6 之外另发现的 `rw` 匹配坑。
221 -/
222
223 /-- 列切片成员性:  $r \in colSlice\ n\ j\ s \leftrightarrow (r, j) \in s$  (定义直接展开)。 -/
224 theorem br_mem_colSlice (n : ℕ) (j : Fin n) (s : Finset (Fin 3 × Fin n))
225   (r : Fin 3) :
226   r ∈ colSlice n j s  $\leftrightarrow$  (r, j) ∈ s := by
227     constructor
228     · intro h
229       have h' : r ∈ univ.filter (fun r => (r, j) ∈ s) := h
230       exact (Finset.mem_filter.mp h').2
231     · intro h
232       exact Finset.mem_filter.mpr ⟨Finset.mem_univ r, h⟩

```

```

228
229 /-- 重建成员性:  $\langle r, j \rangle \in \text{reconstruct } n \ f \leftrightarrow r \in f \ j$  (定义直接展开)。 -/
230 theorem br_mem_reconstruct (n : ℕ) (f : Fin n → Finset (Fin 3)) (r : Fin
231   3) (j : Fin n) :
232    $\langle r, j \rangle \in \text{reconstruct } n \ f \leftrightarrow r \in f \ j$  := by
233   constructor
234   · intro h
235     have h' :  $\langle r, j \rangle \in \text{univ.biUnion } (\text{fun } j' \Rightarrow (f \ j').\text{image } (\text{fun } r' \Rightarrow (r', j')))$  := h
236     obtain  $\langle j', h_j', \text{himg} \rangle$  := Finset.mem_biUnion.mp h'
237     have himg' :  $\langle r, j \rangle \in (f \ j').\text{image } (\text{fun } r' \Rightarrow (r', j'))$  := himg
238     obtain  $\langle r', hr', \text{heq} \rangle$  := Finset.mem_image.mp himg'
239     obtain  $\langle h_1, h_2 \rangle$  := Prod.mk_inj.mp heq
240     subst h2
241     exact h1 ▸ hr'
242   · intro hr
243     have himg :  $\langle r, j \rangle \in (f \ j).\text{image } (\text{fun } r' \Rightarrow (r', j))$  :=
244       Finset.mem_image.mpr  $\langle r, hr, \text{rfl} \rangle$ 
245     exact Finset.mem_biUnion.mpr  $\langle j, \text{Finset.mem_univ } j, \text{himg} \rangle$ 
246 /-- 缺陷集成员性 (中行点)。 -/
247 theorem br_mem_holes_iff (n : ℕ) (j : Fin n) :
248    $((1 : \text{Fin } 3), j) \in \text{holes } 3 \ n \leftrightarrow j.\text{val} \% 2 = 1$  := by
249   simp [holes, Finset.mem_filter]
250
251 /-- 缺陷集成员性 (一般行)。 -/
252 theorem br_mem_holes_iff_row (n : ℕ) (r : Fin 3) (j : Fin n) :
253    $\langle r, j \rangle \in \text{holes } 3 \ n \leftrightarrow r.\text{val} = 1 \wedge j.\text{val} \% 2 = 1$  := by
254   simp [holes, Finset.mem_filter]
255
256 /-- 盒积邻接·竖直分量 (逐字即 SimpleGraph.boxProd_adj_left)。 -/
257 theorem br_adj_vert {n : ℕ} {r r' : Fin 3} {j : Fin n} :
258    $(\text{pathGraph } 3 \sqcap \text{pathGraph } n).\text{Adj } (r, j) (r', j) \leftrightarrow (\text{pathGraph } 3).\text{Adj } r \ r'$ 
259   :=
260   boxProd_adj_left
261
262 /-- 盒积邻接·水平分量 (逐字即 SimpleGraph.boxProd_adj_right)。 -/
263 theorem br_adj_horiz {n : ℕ} {r : Fin 3} {j j' : Fin n} :
264    $(\text{pathGraph } 3 \sqcap \text{pathGraph } n).\text{Adj } (r, j) (r, j') \leftrightarrow (\text{pathGraph } n).\text{Adj } j \ j'$ 
265   :=
266   boxProd_adj_right
267
268 /-- IsIndepSet (Set.Pairwise 形) 改写为干净的  $\forall \forall$  否定形。
269 mathlib v4.34 中 Set.Pairwise s r 带  $x \neq y$  前提 (Logic/Pairwise.lean : 83) ,
270 而 SimpleGraph.ne_of_adj 总能把该前提免费补上。 -/
271 theorem br_isIndepSet_iff_forall {m n : ℕ} (s : Finset (Fin m × Fin n)) :
272   :
273    $(\text{pathGraph } m \sqcap \text{pathGraph } n).\text{IsIndepSet } s \leftrightarrow$ 

```

```

271   ∀ a b : Fin m × Fin n, a ∈ s → b ∈ s → ¬(pathGraph m □
    pathGraph n).Adj a b := by
272 constructor
273 · intro h a b ha hb habj
274   exact h (Finset.mem_coe.mpr ha) (Finset.mem_coe.mpr hb) (ne_of_adj _
    habj) habj
275 · intro h
276   exact fun a ha b hb _ habj => h a b (Finset.mem_coe.mp ha) (Finset.
    mem_coe.mp hb) habj
277
278 /-!
279 ### A2. 竖直独立剖面的有限枚举 ( `Pprof` / `Qprof` 小引理)
280
281 `Finset (Fin 3)` 只有 8 个子集; 枚举入口用 `fin_cases` (实测 `
    interval_cases`
282 不支持 `Fin 3`, 见 notes) 与 5/4 路 `Finset.mem_insert` 链。
283 -/
284
285 /-- 不含中行 ⇒ 属于奇列允许集 `Qprof`。 -/
286 theorem br_Qprof_of_not_mem_one (t : Finset (Fin 3)) (h : (1 : Fin 3) ∉ t
    ) :
287   t ∈ Qprof := by
288   have hsub : t ⊆ ({0, 2} : Finset (Fin 3)) := by
289     intro r hr
290     fin_cases r
291     · exact Finset.mem_insert.2 (Or.inl rfl)
292     · exact absurd hr h
293     · exact Finset.mem_insert.2 (Or.inr (Finset.mem_singleton.2 rfl))
294   have hps : ({0, 2} : Finset (Fin 3)).powerset = Qprof := by decide
295   rw [← hps]
296   exact Finset.mem_powerset.mpr hsub
297
298 /-- 奇列允许集的成员不含中行。 -/
299 theorem br_Qprof_not_mem_one (t : Finset (Fin 3)) (h : t ∈ Qprof) : (1 :
    Fin 3) ∉ t := by
300   rw [Qprof, Finset.mem_insert, Finset.mem_insert, Finset.mem_insert,
    Finset.mem_singleton] at h
301   rcases h with h | h | h | h <;> subst h <;> decide
302
303 /-- 列内无竖直相邻 ⇒ 属于偶列允许集 `Pprof`。 -/
304 theorem br_Pprof_of_indep (t : Finset (Fin 3))
    (h : ∀ r r' : Fin 3, (pathGraph 3).Adj r r' → r ∈ t → r' ∈ t →
    False) :
305   t ∈ Pprof := by
306   by_cases h1 : (1 : Fin 3) ∈ t
307   · have h0 : (0 : Fin 3) ∉ t := fun h0' => h 0 1 (by decide) h0' h1
308     have h2 : (2 : Fin 3) ∉ t := fun h2' => h 1 2 (by decide) h1 h2'
309     have hsub : t ⊆ ({(1 : Fin 3)} : Finset (Fin 3)) := by
310       intro r hr

```



```

358
359 theorem br_allowed_odd (j : ℕ) (h : j % 2 = 1) : allowed j = Qprof := by
360   simp [allowed, h]
361
362 theorem br_allowed_even (j : ℕ) (h : j % 2 ≠ 1) : allowed j = Pprof :=
363   by
364     simp [allowed, h]
365
366 /-!
367 ### A4. 双射两条互逆方向 (guide §3.1 条 1、2)
368 -/
369 /-- 桥梁方向一:  $\psi(\varphi s) = s$ , 无需任何假设 (guide §3.1 条 1)。 -/
370 theorem br_reconstruct_colSlice (n : ℕ) (s : Finset (Fin 3 × Fin n)) :
371   reconstruct n (fun j => colSlice n j s) = s := by
372     apply Finset.ext
373     intro p
374     obtain ⟨r, j⟩ := p
375     constructor
376     · intro h
377       exact (br_mem_colSlice n j s r).mp
378         ((br_mem_reconstruct n (fun j => colSlice n j s) r j).mp h)
379     · intro h
380       exact (br_mem_reconstruct n (fun j => colSlice n j s) r j).mpr
381         ((br_mem_colSlice n j s r).mpr h)
382
383 /-- 桥梁方向二:  $\varphi(\psi f) = f$ , 只用到 `Prod.mk.injEq`, 不需要 `loopless` (
384   guide §3.1 条 2)。 -/
385 theorem br_colSlice_reconstruct (n : ℕ) (f : Fin n → Finset (Fin 3)) (j
386   : Fin n) :
387   colSlice n j (reconstruct n f) = f j := by
388     apply Finset.ext
389     intro r
390     exact (br_mem_colSlice n j (reconstruct n f) r).trans (
391       br_mem_reconstruct n f r j)
392
393 /-!
394 ### A5. 桥梁方向三、四 (合法剖面 ↔ 独立集)
395 -/
396 /-- 桥梁方向三: 与 `holes` 不交的独立集 ⇒ 合法剖面 (guide §3.1 条 3)。 -/
397 theorem br_ValidProf_of_mem_S1 (n : ℕ) (s : Finset (Fin 3 × Fin n)) (h :
398   s ∈ br_S1 n) :
399   ValidProf n (fun j => colSlice n j s) := by
400     obtain ⟨hd, hi⟩ := (Finset.mem_filter.mp h).2
401     have hi' : ∀ a b : Fin 3 × Fin n, a ∈ s → b ∈ s →
402       ¬(pathGraph 3 ⊆ pathGraph n).Adj a b := (br_isIndepSet_iff_forall
403         s).mp hi
404     refine ⟨?, ?_⟩

```

```

401 · intro j
402   show colSlice n j s ∈ allowed j.val
403   by_cases hodd : j.val % 2 = 1
404   · rw [br_allowed_odd j.val hodd]
405     refine br_Qprof_of_not_mem_one _ ?_
406     intro h1
407     have hmem : ((1 : Fin 3), j) ∈ s := (br_mem_colSlice n j s 1).mp h1
408     have hnot : ((1 : Fin 3), j) ∉ holes 3 n := Finset.disjoint_left.mp
      hd hmem
409     exact hnot ((br_mem_holes_iff n j).mpr hodd)
410   · rw [br_allowed_even j.val hodd]
411     refine br_Pprof_of_indep _ ?_
412     intro r r' hadj hr hr'
413     exact hi' (r, j) (r', j) ((br_mem_colSlice n j s r).mp hr)
414       ((br_mem_colSlice n j s r').mp hr') ((br_adj_vert).mpr hadj)
415   · intro j j' hadj
416     rw [Finset.disjoint_left]
417     intro r hrj hrj'
418     exact hi' (r, j) (r, j') ((br_mem_colSlice n j s r).mp hrj)
419       ((br_mem_colSlice n j' s r).mp hrj') ((br_adj_horiz).mpr hadj)
420
421 /-- 桥梁方向四: 合法剖面  $\Rightarrow$  与 holes 不交的独立集 (guide §3.1 条 4)。 -/
422 theorem br_mem_S1_of_ValidProf (n : ℕ) (f : Fin n → Finset (Fin 3)) (h :
  ValidProf n f) :
423   reconstruct n f ∈ br_S1 n := by
424   obtain ⟨hcol, hadjcol⟩ := h
425   refine br_mem_S1 n (reconstruct n f) ?_ ?_
426   · rw [Finset.disjoint_left]
427     intro p hp
428     obtain ⟨r, j⟩ := p
429     rw [br_mem_reconstruct] at hp
430     by_contra hcon
431     rw [br_mem_holes_iff_row] at hcon
432     obtain ⟨hval, hodd⟩ := hcon
433     have hr1 : r = (1 : Fin 3) := by
434       have : r.val = ((1 : Fin 3)).val := by rw [hval]; rfl
435       exact Fin.val_injective this
436     subst hr1
437     have hcolj : f j ∈ allowed j.val := hcol j
438     rw [br_allowed_odd j.val hodd] at hcolj
439     exact br_Qprof_not_mem_one _ hcolj hp
440   · rw [br_isIndepSet_iff_forall]
441     intro a b ha hb habj
442     obtain ⟨ra, ja⟩ := a
443     obtain ⟨rb, jb⟩ := b
444     rw [br_mem_reconstruct] at ha hb
445     rcases boxProd_adj.mp habj with ⟨hvert, hj⟩ | ⟨hhoriz, hr⟩
446     · -- 竖直邻接: 同列相邻行
447       have hvert' : (pathGraph 3).Adj ra rb := hvert

```

```

448     have hj' : ja = jb := hj
449     rw [← hj'] at hb
450     have hP : f ja ∈ Pprof := by
451       have hcolj : f ja ∈ allowed ja.val := hcol ja
452       by_cases hodd : ja.val % 2 = 1
453       · rw [br_allowed_odd ja.val hodd] at hcolj
454       have hQP : Qprof ⊆ Pprof := by decide
455       exact hQP hcolj
456       · rw [br_allowed_even ja.val hodd] at hcolj
457       exact hcolj
458     exact br_Pprof_vert_indep _ hP ra rb hvrt' ha hb
459   · -- 水平邻接: 同行相邻列
460     have hhoriz' : (pathGraph n).Adj ja jb := hhoriz
461     have hr' : ra = rb := hr
462     rw [hr'] at ha
463     have hdis : Disjoint (f ja) (f jb) := hadjcol ja jb hhoriz'
464     exact Finset.disjoint_left.mp hdis ha hb
465
466   /-!
467   ### A6. 计数相等 (`Finset.card_bij`, guide §3.2)
468   -/
469
470   /-- 桥梁主引理: `z n = validCount n` (独立集 ↔ 合法剖面函数的双射)。 -/
471   theorem br_z_eq_validCount (n : ℕ) : z n = validCount n := by
472     rw [br_z_eq_card_S1, br_validCount_eq_card_S2]
473     refine Finset.card_bij (fun s _ => fun j => colSlice n j s) ?_ ?_ ?_
474     · intro s hs
475       exact br_mem_S2 n _ (br_ValidProf_of_mem_S1 n s hs)
476     · intro s1 hs1 s2 hs2 heq
477       have h1 : reconstruct n (fun j => colSlice n j s1)
478         = reconstruct n (fun j => colSlice n j s2) := congrArg (
479         reconstruct n) heq
479       rw [br_reconstruct_colSlice, br_reconstruct_colSlice] at h1
480       exact h1
481     · intro b hb
482       have hbv : ValidProf n b := (Finset.mem_filter.mp hb).2
483       refine ⟨reconstruct n b, br_mem_S1_of_ValidProf n b hbv, ?_⟩
484       funext j
485       exact br_colSlice_reconstruct n b j
486
487   -- =====
488   -- 段 B: 转移 validCount n = 矩阵表达式 (guide §4)
489   -- =====
490
491   /-- 转移侧辅助: 前 `m + 1` 列合法填法数, 且最后一列剖面恰为 `p` (guide §4.2)
492   )。 -/
493   def tr_fillEnd : ℕ → Finset (Fin 3) → ℕ
494     | 0, _ => 0
495     | m + 1, p =>

```

```

495 ((univ : Finset (Fin (m + 1) → Finset (Fin 3))).filter
496   (fun f => ValidProf (m + 1) f ∧ f (Fin.last m) = p)).card
497
498 /-- 转移侧辅助: 偶侧向量  $\text{Vv } k = \text{Tmat} \wedge k \text{ v} * \text{ones5}$  的递归形式 (guide §4.2
    ;
499 记号按 §6 用  $\text{v} * \text{`}$  ( $\text{mulVec}$ , 矩阵在左), guide 正文的  $\text{v} * \text{`}$  为同义简写)。 -/
500 def tr_Vv : ℕ → Fin 5 → ℤ
501   | 0 => fun _ => 1
502   | k + 1 => Tmat v * tr_Vv k
503
504 /-! ### Fin 列指标的分解小引理 -/
505
506 /--  $\text{Fin } (n + 1)$  的元素要么是某个  $\text{Fin } n$  元素的  $\text{castSucc}$ , 要么是  $\text{Fin.}$ 
    last n`。 -/
507 theorem tr_fin_cases {n : ℕ} (i : Fin (n + 1)) :
508   (∃ j : Fin n, i = j.castSucc) ∨ i = Fin.last n := by
509   rcases Nat.lt_or_ge i.val n with h | h
510   · left
511     exact ⟨⟨i.val, h⟩, Fin.ext (by simp)⟩
512   · right
513     exact Fin.ext (by simp only [Fin.val_last]; have := i.isLt; omega)
514
515 /--  $\text{pathGraph}$  邻接在  $\text{castSucc}$  下保持。 -/
516 theorem tr_adj_castSucc {n : ℕ} (i j : Fin n) :
517   (pathGraph (n + 1)).Adj (i.castSucc) (j.castSucc) ↔ (pathGraph n).
    Adj i j := by
518   rw [pathGraph_adj, pathGraph_adj, Fin.val_castSucc, Fin.val_castSucc]
519
520 /--  $\text{Fin } (m + 2)$  的末两列相邻。 -/
521 theorem tr_adj_last_pair {m : ℕ} :
522   (pathGraph (m + 1 + 1)).Adj ((Fin.last m).castSucc) (Fin.last (m + 1))
    := by
523   rw [pathGraph_adj, Fin.val_castSucc]
524   simp only [Fin.val_last]
525   exact Or.inl trivial
526
527 /--  $\text{Fin } (m + 2)$  中与末列相邻的只能是倒数第二列。 -/
528 theorem tr_adj_last_neighbor {m : ℕ} {i : Fin (m + 1)}
529   (h : (pathGraph (m + 1 + 1)).Adj (i.castSucc) (Fin.last (m + 1))) : i
    = Fin.last m := by
530   rw [pathGraph_adj, Fin.val_castSucc] at h
531   simp only [Fin.val_last] at h
532   have hi := i.isLt
533   rcases h with h1 | h1
534   · exact Fin.ext (by simp only [Fin.val_last]; omega)
535   · exfalse; omega
536
537 /--  $\text{Fin } 1$  的唯一元素就是  $\text{Fin.last } 0$ 。 -/
538 theorem tr_fin_one_eq (i : Fin (0 + 1)) : i = Fin.last 0 :=

```

```

539   Fin.ext (by simp only [Fin.val_last]; have := i.isLt; omega)
540
541   /-! ### B1 / B2: 剖面枚举 -/
542
543   /-- B1a: `P5` 枚举恰为 Pprof. -/
544   theorem tr_P5_image : Finset.image P5 univ = Pprof := by
545     decide
546
547   /-- B1b: `P5` 单射. -/
548   theorem tr_P5_inj : Function.Injective P5 := by
549     decide
550
551   /-- B2a: `Q4` 枚举恰为 Qprof. -/
552   theorem tr_Q4_image : Finset.image Q4 univ = Qprof := by
553     decide
554
555   /-- B2b: `Q4` 单射. -/
556   theorem tr_Q4_inj : Function.Injective Q4 := by
557     decide
558
559   /-! ### B3: allowed 的奇偶 -/
560
561   /-- B3a: 偶列允许集 = Pprof. -/
562   theorem tr_allowed_double (j : ℕ) : allowed (2 * j) = Pprof := by
563     simp [allowed]
564
565   /-- B3b: 奇列允许集 = Qprof. -/
566   theorem tr_allowed_double_succ (j : ℕ) : allowed (2 * j + 1) = Qprof :=
567     by
568       simp [allowed]
569
570   /-! ### B4: 单列基例 -/
571
572   /-- `ValidProf 1` 只约束唯一那一列. -/
573   theorem tr_ValidProf_one (f : Fin (0 + 1) → Finset (Fin 3)) :
574     ValidProf (0 + 1) f ↔ f (Fin.last 0) ∈ allowed 0 := by
575     constructor
576     · intro h
577       exact h.1 (Fin.last 0)
578     · intro h
579       refine ⟨fun j => ?_, fun j j' hj => ?_⟩
580       · have hji := tr_fin_one_eq j
581         rw [hji]
582         exact h
583       · exfalso
584         rw [pathGraph_adj] at hj
585         have hj1 := j.isLt
586         have hj2 := j'.isLt
587         omega

```

```

587
588 /-- B4: 单列递推基例。 -/
589 theorem tr_fillEnd_one (p : Finset (Fin 3)) :
590   tr_fillEnd 1 p = if p ∈ allowed 0 then 1 else 0 := by
591   by_cases hp : p ∈ allowed 0
592   · have hif : (if p ∈ allowed 0 then 1 else 0) = 1 := by simp [hp]
593     rw [hif]
594     have h1 : tr_fillEnd 1 p
595       = ((univ : Finset (Fin (0 + 1) → Finset (Fin 3))).filter
596         (fun f => ValidProf (0 + 1) f ∧ f (Fin.last 0) = p)).card :=
597       rfl
598     rw [h1]
599     have h2 : ((univ : Finset (Fin (0 + 1) → Finset (Fin 3))).filter
600       (fun f => ValidProf (0 + 1) f ∧ f (Fin.last 0) = p))
601       = {fun _ : Fin (0 + 1) => p} := by
602       ext f
603       simp only [Finset.mem_filter, Finset.mem_univ, true_and, Finset.
604         mem_singleton]
605       rw [tr_ValidProf_one]
606       constructor
607       · rintro ⟨h1, h2⟩
608         funext i
609         have hii := tr_fin_one_eq i
610         rw [hii]
611         exact h2
612       · rintro rfl
613         exact ⟨hp, rfl⟩
614     rw [h2, Finset.card_singleton]
615   · have hif : (if p ∈ allowed 0 then 1 else 0) = 0 := by simp [hp]
616     rw [hif]
617     have h1 : tr_fillEnd 1 p
618       = ((univ : Finset (Fin (0 + 1) → Finset (Fin 3))).filter
619         (fun f => ValidProf (0 + 1) f ∧ f (Fin.last 0) = p)).card :=
620       rfl
621     rw [h1]
622     have h2 : ((univ : Finset (Fin (0 + 1) → Finset (Fin 3))).filter
623       (fun f => ValidProf (0 + 1) f ∧ f (Fin.last 0) = p)) = ∅ := by
624       ext f
625       simp only [Finset.mem_filter, Finset.mem_univ, true_and, Finset.
626         notMem_empty]
627       exact ⟨fun h => hp (h.2 ▶ h.1.1 (Fin.last 0)), fun h => h.elim⟩
628     rw [h2, Finset.card_empty]
629
630 /-! ### B5: 填法数的双步递推 -/
631 /-- 砍掉末列: `ValidProf (m + 2) f` 给出前 `m + 1` 列的 `ValidProf (m + 1)
632   (Fin.init f)`。 -/
633 theorem tr_validProf_init {m : ℕ} (f : Fin (m + 1 + 1) → Finset (Fin 3))
634   (h : ValidProf (m + 1 + 1) f) : ValidProf (m + 1) (Fin.init f) := by

```

```

631 refine (fun i => ?_, fun i i' hi => ?_)
632 · exact h.1 (i.castSucc)
633 · exact h.2 (i.castSucc) (i'.castSucc) ((tr_adj_castSucc i i').2 hi)
634
635 /-- 接上末列: `Fin.snoc g p` 合法当且仅当 `g` 合法、`p` 在末列允许集、且与倒
    数第二列不交。 -/
636 theorem tr_validProf_snoc {m : ℕ} (g : Fin (m + 1) → Finset (Fin 3)) (p
    : Finset (Fin 3))
637 (hg : ValidProf (m + 1) g) (hp : p ∈ allowed (m + 1))
638 (hdisj : Disjoint (g (Fin.last m)) p) :
639 ValidProf (m + 1 + 1) (Fin.snoc (α := fun _ => Finset (Fin 3)) g p)
    := by
640 refine (fun j => ?_, fun j j' hadj => ?_)
641 · rcases tr_fin_cases j with ⟨i, rfl⟩ | hlast
642 · rw [Fin.snoc_castSucc]
643   exact hg.1 i
644 · rw [hlast, Fin.snoc_last, Fin.val_last]
645   exact hp
646 · rcases tr_fin_cases j with ⟨i, rfl⟩ | hj
647 · rcases tr_fin_cases j' with ⟨i', rfl⟩ | hj'
648 · rw [Fin.snoc_castSucc, Fin.snoc_castSucc]
649   exact hg.2 i i' ((tr_adj_castSucc i i').mp hadj)
650 · rw [hj'] at hadj
651   rw [hj', Fin.snoc_last, Fin.snoc_castSucc]
652   have hi : i = Fin.last m := tr_adj_last_neighbor hadj
653   rw [hi]
654   exact hdisj
655 · rcases tr_fin_cases j' with ⟨i', rfl⟩ | hj'
656 · rw [hj'] at hadj
657   rw [hj, Fin.snoc_last, Fin.snoc_castSucc]
658   have hi' : i' = Fin.last m := tr_adj_last_neighbor (Adj.symm hadj)
659 )
660   rw [hi']
661   refine Finset.disjoint_left.mpr fun r hr hr' => ?_
662   exact Finset.disjoint_left.mp hdisj hr' hr
663 · exfalse
664   rw [hj, hj', pathGraph_adj] at hadj
665   simp only [Fin.val_last] at hadj
666   omega
667 /-- B5: 填法数的双步递推 (先走奇列 Amat 加权, 再走偶列 Bmat 加权)。 -/
668 theorem tr_fillEnd_step (m : ℕ) (p : Finset (Fin 3)) :
669   tr_fillEnd (m + 2) p
670   = if p ∈ allowed (m + 1) then
671     ∑ q ∈ allowed m, (if Disjoint q p then 1 else 0) * tr_fillEnd
672     (m + 1) q
673   else 0 := by
674   have hL : tr_fillEnd (m + 2) p
675     = ((univ : Finset (Fin (m + 1 + 1) → Finset (Fin 3))).filter

```

```

675         (fun f => ValidProf (m + 1 + 1) f ∧ f (Fin.last (m + 1)) = p)).
        card := rfl
676   rw [hL]
677   split_ifs with hp
678   · -- 主情形：末列剖面  $p$  合法
679     have hcard : ((univ : Finset (Fin (m + 1 + 1) → Finset (Fin 3))).
        filter
680       (fun f => ValidProf (m + 1 + 1) f ∧ f (Fin.last (m + 1)) = p)).
        card
681       = ((allowed m).biUnion (fun q => (univ : Finset (Fin (m + 1) →
        Finset (Fin 3))).filter
682         (fun g => ValidProf (m + 1) g ∧ g (Fin.last m) = q ∧ Disjoint
        q p))).card := by
683       apply Finset.card_bij (i := fun f _ => Fin.init f)
684       · -- 映⼊
685         intro f hf
686         obtain ⟨-, hval, hfp⟩ := Finset.mem_filter.mp hf
687         refine Finset.mem_biUnion.mpr ⟨f ((Fin.last m).castSucc), ?_, ?_⟩
688         · exact hval.1 ((Fin.last m).castSucc)
689         · refine Finset.mem_filter.mpr ⟨Finset.mem_univ _, ?_⟩
690           refine ⟨tr_validProf_init f hval, rfl, ?_⟩
691           exact hfp ▶ hval.2 ((Fin.last m).castSucc) (Fin.last (m + 1))
        tr_adj_last_pair
692       · -- 单射
693         intro f1 hf1 f2 hf2 h
694         obtain ⟨-, hv1, hl1⟩ := Finset.mem_filter.mp hf1
695         obtain ⟨-, hv2, hl2⟩ := Finset.mem_filter.mp hf2
696         calc f1 = Fin.snoc (α := fun _ => Finset (Fin 3)) (Fin.init f1) (
        f1 (Fin.last (m + 1))) :=
697           (Fin.snoc_init_self f1).symm
698         _ = Fin.snoc (α := fun _ => Finset (Fin 3)) (Fin.init f2) (f2 (
        Fin.last (m + 1))) := by
699           rw [h]
700           _ = Fin.snoc (α := fun _ => Finset (Fin 3)) (Fin.init f2) (f2 (
        Fin.last (m + 1))) := by
701             rw [hl1, hl2]
702             _ = f2 := Fin.snoc_init_self f2
703       · -- 满射
704         intro b hb
705         obtain ⟨q, hqm, hbf⟩ := Finset.mem_biUnion.mp hb
706         obtain ⟨-, hbval, hblast, hbdisj⟩ := Finset.mem_filter.mp hbf
707         refine ⟨Fin.snoc (α := fun _ => Finset (Fin 3)) b p, ?_, ?_⟩
708         · refine Finset.mem_filter.mpr ⟨Finset.mem_univ _, ?_⟩
709           refine ⟨tr_validProf_snoc b p hbval hp (hblast ▶ hbdisj),
710             Fin.snoc_last (α := fun _ => Finset (Fin 3)) p b⟩
711         · exact Fin.init_snoc (α := fun _ => Finset (Fin 3)) p b
712   rw [hcard, Finset.card_biUnion]
713   · refine Finset.sum_congr rfl fun q hq => ?_
714   split_ifs with hd

```

```

715 · rw [one_mul]
716   have hfib : (univ : Finset (Fin (m + 1) → Finset (Fin 3))).
filter
717   (fun g => ValidProf (m + 1) g ∧ g (Fin.last m) = q ∧ Disjoint
q p)
718   = (univ : Finset (Fin (m + 1) → Finset (Fin 3))).filter
719   (fun g => ValidProf (m + 1) g ∧ g (Fin.last m) = q) := by
720   ext g
721   simp only [Finset.mem_filter, Finset.mem_univ, true_and, hd,
and_true]
722   rw [hfib]
723   rfl
724 · have hfib : (univ : Finset (Fin (m + 1) → Finset (Fin 3))).filter
725   (fun g => ValidProf (m + 1) g ∧ g (Fin.last m) = q ∧ Disjoint
q p) = ∅ := by
726   ext g
727   simp only [Finset.mem_filter, Finset.mem_univ, hd, and_false]
728   exact ⟨fun h => h.elim, fun h => Finset.notMem_empty g h⟩
729   rw [hfib, Finset.card_empty, zero_mul]
730 · -- 纤维两两不交
731   intro a ha b hb hab x hxa hxb y hy
732   exfalse
733   apply hab
734   have h1 : y (Fin.last m) = a := (((Finset.mem_filter.mp (hxa hy)).
right).right).left
735   have h2 : y (Fin.last m) = b := (((Finset.mem_filter.mp (hxb hy)).
right).right).left
736   rw [← h1, ← h2]
737 · -- 末列剖面不允许: filter 为空
738   have hempty : (univ : Finset (Fin (m + 1 + 1) → Finset (Fin 3))).
filter
739   (fun f => ValidProf (m + 1 + 1) f ∧ f (Fin.last (m + 1)) = p) = ∅
:= by
740   ext f
741   simp only [Finset.mem_filter, Finset.mem_univ, true_and, Finset.
notMem_empty, iff_false]
742   rintro ⟨hval, hfp⟩
743   exact hp (hfp ▶ hval.1 (Fin.last (m + 1)))
744   rw [hempty, Finset.card_empty]
745
746 /-! ### B6: 零列恰一合法填法 -/
747
748 /-- B6: 零列恰一合法填法。 -/
749 theorem tr_validCount_zero : validCount 0 = 1 := by
750   have hval : ∀ f : Fin 0 → Finset (Fin 3), ValidProf 0 f := fun f =>
751   ⟨fun j => Fin.elim0 j, fun j j' _ => Fin.elim0 j⟩
752   have hfil : (univ : Finset (Fin 0 → Finset (Fin 3))).filter (ValidProf
0) = univ := by
753   ext f

```

```

754   simp [hval]
755   have hsub : Subsingleton (Fin 0 → Finset (Fin 3)) :=
756     Subsingleton.intro fun a b => by
757       funext j
758       exact Fin.elim0 j
759   have hcard : (univ : Finset (Fin 0 → Finset (Fin 3))).card = 1 := by
760     have h1 : (univ : Finset (Fin 0 → Finset (Fin 3))).card ≤ 1 :=
761       Finset.card_le_one.mpr fun a _ b _ => Subsingleton.elim a b
762     have h2 : 0 < (univ : Finset (Fin 0 → Finset (Fin 3))).card :=
763       Finset.card_pos.mpr Finset.univ_nonempty
764     omega
765   simp only [validCount]
766   rw [hfil, hcard]
767
768   /-! ### B7: 合法剖面数 = 按末列剖面求和 -/
769
770   /-- B7: 合法剖面数 = 按末列剖面求和。 -/
771   theorem tr_validCount_succ (m : ℕ) :
772     validCount (m + 1) =  $\sum p \in \text{allowed } m, \text{tr\_fillEnd } (m + 1) p$  := by
773     have hfib : (univ : Finset (Fin (m + 1) → Finset (Fin 3))).filter (
774       ValidProf (m + 1))
775       = (allowed m).biUnion (fun p => (univ : Finset (Fin (m + 1) →
776         Finset (Fin 3))).filter
777         (fun f => ValidProf (m + 1) f ∧ f (Fin.last m) = p)) := by
778       ext f
779       simp only [Finset.mem_filter, Finset.mem_biUnion, Finset.mem_univ,
780         true_and]
781       constructor
782       · intro h
783         refine ⟨f (Fin.last m), h.1 (Fin.last m), h, rfl⟩
784       · rintro ⟨p, -, hf, hfp⟩
785         exact hf
786     rw [validCount, hfib]
787     rw [Finset.card_biUnion]
788     · rfl
789     · intro a ha b hb hab x hxa hxb y hy
790       exfalse
791       apply hab
792       have h1 : y (Fin.last m) = a := ((Finset.mem_filter.mp (hxa hy)).
793         right).right
794       have h2 : y (Fin.last m) = b := ((Finset.mem_filter.mp (hxb hy)).
795         right).right
796       rw [← h1, ← h2]
797
798   /-! ### B8 / B9 的辅助: 枚举成员性、cast 处理、if 权重对齐、联合归纳 -/
799
800   /-- 用可判定条件的证明化简 `if` (取真支)。 -/
801   theorem tr_if_pos {α : Type} {c : Prop} [Decidable c] (h : c) (a b : α) :
802     (if c then a else b) = a := by

```

```

798   simp [h]
799
800 /--  $\mathbb{N} \rightarrow \mathbb{Z}$  的 cast 穿过 if。 -/
801 theorem tr_cast_if {c : Prop} [Decidable c] (a b :  $\mathbb{N}$ ) :
802   ((if c then a else b :  $\mathbb{N}$ ) :  $\mathbb{Z}$ ) = if c then (a :  $\mathbb{Z}$ ) else (b :  $\mathbb{Z}$ ) :=
803   by
804     by_cases h : c
805     · simp [h]
806     · simp [h]
807 /--  $\mathbb{N} \rightarrow \mathbb{Z}$  的 cast 穿过 (if c then 1 else 0) * f (一步到位)。 -/
808 theorem tr_cast_ite_mul {c : Prop} [Decidable c] (f :  $\mathbb{N}$ ) :
809   (((if c then 1 else 0) * f :  $\mathbb{N}$ ) :  $\mathbb{Z}$ ) = (if c then (1 :  $\mathbb{Z}$ ) else 0) *
810   (f :  $\mathbb{Z}$ ) := by
811     by_cases h : c
812     · simp [h]
813     · simp [h]
814 /-- P5 的每个分量都在 Pprof 中。 -/
815 theorem tr_P5_mem (i : Fin 5) : P5 i ∈ Pprof := by
816   rw [← tr_P5_image]
817   exact Finset.mem_image.mpr ⟨i, Finset.mem_univ i, rfl⟩
818 /-- Q4 的每个分量都在 Qprof 中。 -/
819 theorem tr_Q4_mem (j : Fin 4) : Q4 j ∈ Qprof := by
820   rw [← tr_Q4_image]
821   exact Finset.mem_image.mpr ⟨j, Finset.mem_univ j, rfl⟩
822 /-- P5 在 univ 上单射 (sum_image 用)。 -/
823 theorem tr_P5_injOn : Set.InjOn P5 (↑(univ : Finset (Fin 5))) := by
824   intro a _ b _ h
825   exact tr_P5_inj h
826 /-- Q4 在 univ 上单射 (sum_image 用)。 -/
827 theorem tr_Q4_injOn : Set.InjOn Q4 (↑(univ : Finset (Fin 4))) := by
828   intro a _ b _ h
829   exact tr_Q4_inj h
830 /-- 不交 if 权重交换两参数后相同 ( $\mathbb{Z}$  系数版)。 -/
831 theorem tr_ite_disjoint_comm (a b : Finset (Fin 3)) :
832   (if Disjoint a b then (1 :  $\mathbb{Z}$ ) else 0) = (if Disjoint b a then (1 :  $\mathbb{Z}$ )
833   else 0) := by
834     by_cases h : Disjoint a b
835     · have h' : Disjoint b a := _root_.disjoint_comm.mp h
836       simp [h, h']
837     · have h' : ¬ Disjoint b a := fun h' => h (_root_.disjoint_comm.mpr h')
838       simp [h, h']
839 /-- Amat i j = Bmat j i (不交条件交换两个剖面)。 -/

```

```

844 theorem tr_Amat_eq_Bmat (i : Fin 5) (j : Fin 4) : Amat i j = Bmat j i :=
      by
845   show (if Disjoint (P5 i) (Q4 j) then (1 : ℤ) else 0)
846   = (if Disjoint (Q4 j) (P5 i) then (1 : ℤ) else 0)
847   exact tr_ite_disjoint_comm (P5 i) (Q4 j)
848
849 /-- 矩阵幂前移一步:  $Tmat^{k+1} = Tmat * Tmat^k$ . -/
850 theorem tr_pow_succ' (k : ℕ) : Tmat ^ (k + 1) = Tmat * Tmat ^ k := by
851   rw [pow_succ]
852   exact (Commute.eq ((Commute.refl Tmat).pow_right k)).symm
853
854 /-- B8/B9 的联合归纳形式 (计数提升到 ℤ 后陈述)。 -/
855 theorem tr_fillEnd_pair (k : ℕ) :
856   (∀ i : Fin 5, (tr_fillEnd (2 * k + 1) (P5 i) : ℤ) = ((Tmat ^ k) _*
      ones5) i) ∧
857   (∀ j : Fin 4, (tr_fillEnd (2 * k + 2) (Q4 j) : ℤ) = (Bmat _* ((Tmat ^
      k) _* ones5)) j) := by
858   induction k with
859   | zero =>
860     refine ⟨?, ?⟩
861     · intro i
862       rw [show 2 * 0 + 1 = 1 from by simp, tr_fillEnd_one, tr_cast_if,
863         show allowed 0 = Pprof from by simp [allowed], tr_if_pos (
      tr_P5_mem i)]
864       rw [pow_zero, Matrix.one_mulVec, Nat.cast_one]
865       rfl
866     · intro j
867       rw [show 2 * 0 + 2 = 0 + 2 from by simp, tr_fillEnd_step,
      tr_cast_if,
868         show (0 : ℕ) + 1 = 1 from by simp, show allowed 1 = Qprof from by
      simp [allowed],
869         show allowed 0 = Pprof from by simp [allowed], tr_if_pos (
      tr_Q4_mem j), Nat.cast_sum]
870       rw [pow_zero, Matrix.one_mulVec]
871       rw [← tr_P5_image, Finset.sum_image tr_P5_injOn]
872       rw [show (Bmat _* ones5) j
873         = ∑ i : Fin 5, (if Disjoint (Q4 j) (P5 i) then (1 : ℤ) else 0)
      * ones5 i from rfl]
874       refine Finset.sum_congr rfl fun i _ => ?_
875       have h1 : tr_fillEnd 1 (P5 i) = 1 := by
876         rw [tr_fillEnd_one, show allowed 0 = Pprof from by simp [allowed]
      ],
877         tr_if_pos (tr_P5_mem i)]
878       rw [tr_cast_ite_mul, h1, Nat.cast_one, mul_one,
879         show ones5 i = (1 : ℤ) from rfl, mul_one]
880       exact tr_ite_disjoint_comm (P5 i) (Q4 j)
881   | succ k ih =>
882     obtain ⟨ihP, ihQ⟩ := ih
883     have hallowed1 : allowed (2 * k + 1 + 1) = Pprof := by

```

```

884     rw [show 2 * k + 1 + 1 = 2 * (k + 1) from by omega]
885     exact tr_allowed_double (k + 1)
886   have hallowed2 : allowed (2 * k + 2 + 1) = Qprof := by
887     rw [show 2 * k + 2 + 1 = 2 * (k + 1) + 1 from by omega]
888     exact tr_allowed_double_succ (k + 1)
889   have hallowed3 : allowed (2 * k + 2) = Pprof := by
890     rw [show 2 * k + 2 = 2 * (k + 1) from by omega]
891     exact tr_allowed_double (k + 1)
892   have hP1 : ∀ i : Fin 5,
893     (tr_fillEnd (2 * (k + 1) + 1) (P5 i) : ℤ) = ((Tmat ^ (k + 1)) v *
ones5) i := by
894     intro i
895     have key : ((Tmat ^ (k + 1)) v * ones5) i
896       = ∑ j : Fin 4, Bmat j i * (Bmat v * ((Tmat ^ k) v * ones5)) j :=
by
897     have h1 : (Amat v * (Bmat v * ((Tmat ^ k) v * ones5))) = (Tmat v * ((
Tmat ^ k) v * ones5)) :=
898       Matrix.mulVec_mulVec _ Amat Bmat
899     rw [tr_pow_succ', ← Matrix.mulVec_mulVec, ← h1]
900     show (∑ j : Fin 4, Amat i j * (Bmat v * ((Tmat ^ k) v * ones5)) j)
=
901     - exact Finset.sum_congr rfl fun j _ => by rw [tr_Amat_eq_Bmat i j]
902     rw [show 2 * (k + 1) + 1 = 2 * k + 1 + 2 from by omega,
tr_fillEnd_step, tr_cast_if,
903     hallowed1, tr_allowed_double_succ, tr_if_pos (tr_P5_mem i), Nat.
cast_sum, key]
904     rw [← tr_Q4_image, Finset.sum_image tr_Q4_injOn]
905     refine Finset.sum_congr rfl fun j _ => ?_
906     rw [tr_cast_ite_mul, show 2 * k + 1 + 1 = 2 * k + 2 from by omega,
ihQ j]
907     rfl
908   refine ⟨hP1, ?_⟩
909   intro j
910   rw [show (Bmat v * ((Tmat ^ (k + 1)) v * ones5)) j
= ∑ i : Fin 5, (if Disjoint (Q4 j) (P5 i) then (1 : ℤ) else 0)
* ((Tmat ^ (k + 1)) v * ones5) i from rfl]
911   rw [show 2 * (k + 1) + 2 = 2 * k + 2 + 2 from by omega,
tr_fillEnd_step, tr_cast_if,
912     hallowed2, hallowed3, tr_if_pos (tr_Q4_mem j), Nat.cast_sum]
913   rw [← tr_P5_image, Finset.sum_image tr_P5_injOn]
914   refine Finset.sum_congr rfl fun i _ => ?_
915   rw [tr_cast_ite_mul, show 2 * k + 2 + 1 = 2 * (k + 1) + 1 from by
omega, hP1 i,
916     tr_ite_disjoint_comm (P5 i) (Q4 j)]
917
918
919
920 /-- `tr_Vv` 与矩阵幂表示一致 (对  $k$  归纳)。 -/
921 theorem tr_Vv_eq (k : ℕ) : tr_Vv k = (Tmat ^ k) v * ones5 := by
922   induction k with
923   | zero =>

```

```

924     rw [pow_zero, Matrix.one_mulVec]
925     rfl
926   | succ k ih =>
927     rw [tr_Vv, ih, Matrix.mulVec_mulVec, tr_pow_succ']
928
929 /-- B8: 奇列末的填法数 =  $T_{mat}^k \cdot \mathbf{ones}_5$  分量。 -/
930 theorem tr_fillEnd_P5 (k : ℕ) (i : Fin 5) :
931   tr_fillEnd (2 * k + 1) (P5 i) = ((Tmat ^ k) _v* ones5) i :=
932   (tr_fillEnd_pair k).1 i
933
934 /-- B9: 偶列末的填法数 =  $B_{mat} \cdot \mathbf{ones}_5$  分量。 -/
935 theorem tr_fillEnd_Q4 (k : ℕ) (j : Fin 4) :
936   tr_fillEnd (2 * k + 2) (Q4 j) = (Bmat _v* ((Tmat ^ k) _v* ones5)) j :=
937   (tr_fillEnd_pair k).2 j
938
939 /-- B10: 奇宽度计数 =  $\mathbf{ones}_5$  侧矩阵和。 -/
940 theorem tr_validCount_odd (k : ℕ) :
941   validCount (2 * k + 1) =  $\sum i : \text{Fin } 5, ((Tmat ^ k) _v* \mathbf{ones}_5) i$  := by
942   rw [tr_validCount_succ, Nat.cast_sum, tr_allowed_double, ← tr_P5_image
943     ,
944     Finset.sum_image tr_P5_injOn]
945   exact Finset.sum_congr rfl fun i _ => tr_fillEnd_P5 k i
946
947 /-- B11: 偶宽度计数 =  $\mathbf{vec}_0$  侧矩阵和。 -/
948 theorem tr_validCount_even (k : ℕ) :
949   validCount (2 * k + 2) =  $\sum j : \text{Fin } 4, (Bmat \cdot ((Tmat ^ k) \cdot \mathbf{ones}_5)) j$  := by
950   rw [show 2 * k + 2 = (2 * k + 1) + 1 from by omega, tr_validCount_succ,
951     Nat.cast_sum,
952     show (2 * k + 1) + 1 = 2 * k + 2 from by omega,
953     tr_allowed_double_succ, ← tr_Q4_image, Finset.sum_image tr_Q4_injOn]
954   exact Finset.sum_congr rfl fun j _ => tr_fillEnd_Q4 k j
955
956 -- =====
957 -- 段 C: 代数归约 (组装员追加; 新引理统一前缀 `asm_`, 不与 br_ / tr_ 重名)
958 -- 数学路线: attempts/guide.md §5 (证书 → w/u 递推 → 奇偶归约 → 唯一边界
959 --            $n = 6$ )
960 -- =====
961
962 /-!
963 ### C0. 求和  $\sum_v$  的  $\mathbb{Z}$ -线性辅助
964
965 不复用 mathlib 分配律名 (避免名字漂移), 全部自证: Finset.induction`
966 (empty` / insert` 两情形) + Finset.sum_insert` + ring`。
967 asm_mulVec_lin` 经 Matrix.mulVec` (dotProduct` 的点态展开) 把  $M \cdot v$ 
968 搬到分量上, 再套 C0 的求和恒等式。
969 -/
970 /-- 常数倍过求和 (左系数版)。 -/

```

```

969 theorem asm_mul_sum {ι : Type} (s : Finset ι) (c : ℤ) (g : ι → ℤ) :
970   c * ∑ j ∈ s, g j = ∑ j ∈ s, c * g j := by
971   classical
972   induction s using Finset.induction with
973   | empty => simp
974   | insert a s ha ih => rw [Finset.sum_insert ha, Finset.sum_insert ha,
     mul_add, ih]
975
976 /-- 两个求和的加法合并。 -/
977 theorem asm_sum_add {ι : Type} (s : Finset ι) (f g : ι → ℤ) :
978   (∑ j ∈ s, f j) + (∑ j ∈ s, g j) = ∑ j ∈ s, (f j + g j) := by
979   classical
980   induction s using Finset.induction with
981   | empty => simp
982   | insert a s ha ih =>
983     rw [Finset.sum_insert ha, Finset.sum_insert ha, Finset.sum_insert
     ha]
984     calc (f a + ∑ x ∈ s, f x) + (g a + ∑ x ∈ s, g x)
985       = (f a + g a) + (∑ x ∈ s, f x + ∑ x ∈ s, g x) := by ring
986       _ = (f a + g a) + ∑ x ∈ s, (f x + g x) := by rw [ih]
987
988 /-- 两个求和的减法合并。 -/
989 theorem asm_sum_sub {ι : Type} (s : Finset ι) (f g : ι → ℤ) :
990   (∑ j ∈ s, f j) - (∑ j ∈ s, g j) = ∑ j ∈ s, (f j - g j) := by
991   classical
992   induction s using Finset.induction with
993   | empty => simp
994   | insert a s ha ih =>
995     rw [Finset.sum_insert ha, Finset.sum_insert ha, Finset.sum_insert
     ha]
996     calc (f a + ∑ x ∈ s, f x) - (g a + ∑ x ∈ s, g x)
997       = (f a - g a) + (∑ x ∈ s, f x - ∑ x ∈ s, g x) := by ring
998       _ = (f a - g a) + ∑ x ∈ s, (f x - g x) := by rw [ih]
999
1000 /-- ℤ-线性组合 `12 · a - 15 · b + 2 · c` 过求和。 -/
1001 theorem asm_sum_lin {ι : Type} (s : Finset ι) (M : ι → ℤ) (a b c : ι → ℤ) :
1002   ∑ j ∈ s, M j * (12 * a j - 15 * b j + 2 * c j)
1003   = 12 * ∑ j ∈ s, M j * a j - 15 * ∑ j ∈ s, M j * b j + 2 * ∑ j ∈
     s, M j * c j := by
1004   rw [asm_mul_sum, asm_mul_sum, asm_mul_sum, asm_sum_sub, asm_sum_add]
1005   exact Finset.sum_congr rfl fun j _ => by ring
1006
1007 /-!
1008 ### C1. w-递推 / u-递推 (guide §5.1)
1009
1010 `cert_ones` (interface 段三) 给 `k = 0` 的点态递推; `Matrix.mulVec_mulVec`
1011 (向量占第一个显式参数位) + 裸 `pow_add` 把 `Tmat ^ k` 前后搬,
1012 `asm_mulVec_lin` 处理线性组合, 得任意 `k` 的递推。u-递推由 w-递推对

```

```

1013 `Bmat_v*` 逐点搬得到 (guide §5.1 末段)。
1014 -/
1015
1016 /-- `M_v*` 对点态  $\mathbb{Z}$ -线性组合的作用 (分量形)。 -/
1017 theorem asm_mulVec_lin (M : Matrix (Fin 5) (Fin 5)  $\mathbb{Z}$ ) (a b c : Fin 5  $\rightarrow \mathbb{Z}$ )
1018   (i : Fin 5) :
1019   (M_v* fun j => 12 * a j - 15 * b j + 2 * c j) i
1020   = 12 * (M_v* a) i - 15 * (M_v* b) i + 2 * (M_v* c) i := by
1021   show  $\sum j : \text{Fin } 5, M_{ij} * (12 * a_j - 15 * b_j + 2 * c_j)$ 
1022   = 12 * ( $\sum j : \text{Fin } 5, M_{ij} * a_j$ ) - 15 * ( $\sum j : \text{Fin } 5, M_{ij} * b_j$ )
1023   + 2 * ( $\sum j : \text{Fin } 5, M_{ij} * c_j$ )
1024   exact asm_sum_lin (s := (univ : Finset (Fin 5))) (fun j => M i j) a b c
1025
1026 /-- w-递推 (点态) : `w k = Tmat ^ k_v* ones5`。 -/
1027 theorem asm_w_rec (k :  $\mathbb{N}$ ) (i : Fin 5) :
1028   ((Tmat ^ (k + 3))_v* ones5) i
1029   = 12 * ((Tmat ^ (k + 2))_v* ones5) i - 15 * ((Tmat ^ (k + 1))_v*
1030     ones5) i
1031   + 2 * ((Tmat ^ k)_v* ones5) i := by
1032   have hstep : (Tmat ^ (k + 3))_v* ones5 = (Tmat ^ k)_v* ((Tmat ^ 3)_v*
1033     ones5) := by
1034     rw [pow_add]
1035     exact (Matrix.mulVec_mulVec _ _ _).symm
1036   have hcert : (Tmat ^ 3)_v* ones5
1037     = fun j => 12 * ((Tmat ^ 2)_v* ones5) j - 15 * ((Tmat ^ 1)_v* ones5)
1038       j + 2 * ones5 j := by
1039     funext j
1040     exact cert_ones j
1041   have e2 : (Tmat ^ k)_v* ((Tmat ^ 2)_v* ones5) = (Tmat ^ (k + 2))_v*
1042     ones5 := by
1043     rw [Matrix.mulVec_mulVec, ← pow_add]
1044   have e1 : (Tmat ^ k)_v* ((Tmat ^ 1)_v* ones5) = (Tmat ^ (k + 1))_v*
1045     ones5 := by
1046     rw [Matrix.mulVec_mulVec, ← pow_add]
1047   rw [hstep, hcert, asm_mulVec_lin, e2, e1]
1048
1049 /-- u-递推 (点态) : `u k = Bmat_v* (Tmat ^ k_v* ones5)`。 -/
1050 theorem asm_u_rec (k :  $\mathbb{N}$ ) (j : Fin 4) :
1051   (Bmat_v* ((Tmat ^ (k + 3))_v* ones5)) j
1052   = 12 * (Bmat_v* ((Tmat ^ (k + 2))_v* ones5)) j
1053   - 15 * (Bmat_v* ((Tmat ^ (k + 1))_v* ones5)) j + 2 * (Bmat_v* ((
1054     Tmat ^ k)_v* ones5)) j := by
1055   show  $\sum i : \text{Fin } 5, Bmat_{ji} * (((Tmat ^ (k + 3))_v* ones5) i)$ 
1056   = 12 * ( $\sum i : \text{Fin } 5, Bmat_{ji} * (((Tmat ^ (k + 2))_v* ones5) i)$ )
1057   - 15 * ( $\sum i : \text{Fin } 5, Bmat_{ji} * (((Tmat ^ (k + 1))_v* ones5) i)$ )
1058   + 2 * ( $\sum i : \text{Fin } 5, Bmat_{ji} * (((Tmat ^ k)_v* ones5) i)$ )
1059   rw [asm_mul_sum, asm_mul_sum, asm_mul_sum, asm_sum_sub, asm_sum_add]
1060   exact Finset.sum_congr rfl fun i _ => by rw [asm_w_rec k i]; ring

```

```

1054
1055 /-- w-递推（求和形，奇侧用）。 -/
1056 theorem asm_w_rec_sum (k : ℕ) :
1057   (∑ i : Fin 5, ((Tmat ^ (k + 3)) v * ones5) i)
1058   = 12 * (∑ i : Fin 5, ((Tmat ^ (k + 2)) v * ones5) i)
1059   - 15 * (∑ i : Fin 5, ((Tmat ^ (k + 1)) v * ones5) i) + 2 * (∑ i
    : Fin 5, ((Tmat ^ k) v * ones5) i) := by
1060   rw [asm_mul_sum, asm_mul_sum, asm_mul_sum, asm_sum_sub, asm_sum_add]
1061   exact Finset.sum_congr rfl fun i _ => asm_w_rec k i
1062
1063 /-- u-递推（求和形，偶侧用）。 -/
1064 theorem asm_u_rec_sum (k : ℕ) :
1065   (∑ j : Fin 4, (Bmat v * ((Tmat ^ (k + 3)) v * ones5)) j)
1066   = 12 * (∑ j : Fin 4, (Bmat v * ((Tmat ^ (k + 2)) v * ones5)) j)
1067   - 15 * (∑ j : Fin 4, (Bmat v * ((Tmat ^ (k + 1)) v * ones5)) j)
1068   + 2 * (∑ j : Fin 4, (Bmat v * ((Tmat ^ k) v * ones5)) j) := by
1069   rw [asm_mul_sum, asm_mul_sum, asm_mul_sum, asm_sum_sub, asm_sum_add]
1070   exact Finset.sum_congr rfl fun j _ => asm_u_rec k j
1071
1072 /-!
1073 ### C2. 主定理（陈述与 `formalized/statement.lean` 冻结件逐字一致）
1074
1075 按 guide §5.2-5.3 组装：
1076 - 奇 `n = 2m + 1`（`6 ≤ n`  $\implies$  `m ≥ 3`）：走 w-递推，无边界情形；
1077 - 偶 `n = 2m`（`n ≠ 6`  $\implies$  `m ≥ 4`）：走 u-递推；
1078 - 唯一边界 `n = 6`：`z 6 / z 4 / z 2` 走矩阵和、`z 0 = 1` 走 B6，`decide`
    闭。
1079
1080 ℕ/ℤ 口径：`tr_validCount_odd` / `tr_validCount_even` / `
    tr_validCount_zero`
1081 的右端是 ℤ 矩阵和，故每个分支先把要用的等式搬到 ℤ（`have key : (… : ℤ) =
    …`），
1082 分支内用 `omega` 收口；最后 `omega` 借 ℤ 假设回证 ℕ 目标（本机 kernel 实测
1083 `omega` 可桥接 `Nat.cast` 两侧，见 `tmp-probe/comb12-asm-probe.lean` T2
    ）。
1084 `n - 4` / `n - 2` / `n - 6` 是 ℕ 截断减法，用 `omega` 翻译成 `2 * (m - k)
    + c` 形。
1085 -/
1086
1087 theorem c_z_rec : ∀ n : ℕ, 6 ≤ n → z n + 15 * z (n - 4) = 12 * z (n -
    2) + 2 * z (n - 6) := by
1088   intro n hn
1089   rcases eq_or_ne n 6 with rfl | hne
1090   · -- 唯一边界 n = 6
1091     have h4 : (6 - 4 : ℕ) = 2 := by decide
1092     have h2 : (6 - 2 : ℕ) = 4 := by decide
1093     have h0 : (6 - 6 : ℕ) = 0 := by decide
1094     have key : (z 6 : ℤ) + 15 * (z 2 : ℤ) = 12 * (z 4 : ℤ) + 2 * (z 0 :
        ℤ) := by

```

```

1095   have h1 : (z 6 : ℤ) = ∑ j : Fin 4, (Bmat v* ((Tmat ^ 2) v* ones5))
      j := by
1096     have h := tr_validCount_even 2
1097     rw [← br_z_eq_validCount (2 * 2 + 2)] at h
1098     exact h
1099   have h2' : (z 2 : ℤ) = ∑ j : Fin 4, (Bmat v* ((Tmat ^ 0) v* ones5))
      j := by
1100     have h := tr_validCount_even 0
1101     rw [← br_z_eq_validCount (2 * 0 + 2)] at h
1102     exact h
1103   have h3 : (z 4 : ℤ) = ∑ j : Fin 4, (Bmat v* ((Tmat ^ 1) v* ones5))
      j := by
1104     have h := tr_validCount_even 1
1105     rw [← br_z_eq_validCount (2 * 1 + 2)] at h
1106     exact h
1107   have h0z : (z 0 : ℤ) = 1 := by
1108     have h := tr_validCount_zero
1109     rw [← br_z_eq_validCount 0] at h
1110     exact_mod_cast h
1111   have vsum : (∑ j : Fin 4, (Bmat v* ((Tmat ^ 2) v* ones5)) j)
1112     + 15 * (∑ j : Fin 4, (Bmat v* ((Tmat ^ 0) v* ones5)) j)
1113     = 12 * (∑ j : Fin 4, (Bmat v* ((Tmat ^ 1) v* ones5)) j) + 2 *
1 := by
1114   decide
1115   omega
1116   rw [h4, h2, h0]
1117   omega
1118   -- n ≥ 7: 奇偶分岔 (guide §5.2)
1119   have hpar : ∃ m, n = 2 * m ∨ n = 2 * m + 1 := by
1120     refine ⟨n / 2, ?_⟩
1121     omega
1122   obtain ⟨m, hw⟩ := hpar
1123   rcases hw with rfl | rfl
1124   · -- 偶 n = 2 * m, hne : 2 * m ≠ 6 且 6 ≤ 2 * m ⇒ m ≥ 4
1125     have hm : 4 ≤ m := by omega
1126     have e4 : 2 * m - 4 = 2 * (m - 3) + 2 := by omega
1127     have e2 : 2 * m - 2 = 2 * (m - 2) + 2 := by omega
1128     have e6 : 2 * m - 6 = 2 * (m - 4) + 2 := by omega
1129     have e0 : 2 * m = 2 * (m - 1) + 2 := by omega
1130     have key : (z (2 * (m - 1) + 2) : ℤ) + 15 * (z (2 * (m - 3) + 2) :
1131       ℤ)
      = 12 * (z (2 * (m - 2) + 2) : ℤ) + 2 * (z (2 * (m - 4) + 2) : ℤ)
      := by
1132       have h1 : (z (2 * (m - 1) + 2) : ℤ)
1133         = ∑ j : Fin 4, (Bmat v* ((Tmat ^ (m - 1)) v* ones5)) j := by
1134         have h := tr_validCount_even (m - 1)
1135         rw [← br_z_eq_validCount (2 * (m - 1) + 2)] at h
1136         exact h
1137       have h2 : (z (2 * (m - 3) + 2) : ℤ)

```

```

1138      =  $\sum j : \text{Fin } 4, (\text{Bmat}_v * ((\text{Tmat} \wedge (m - 3))_v * \text{ones5})) j := \text{by}$ 
1139      have h := tr_validCount_even (m - 3)
1140      rw [← br_z_eq_validCount (2 * (m - 3) + 2)] at h
1141      exact h
1142      have h3 : (z (2 * (m - 2) + 2) :  $\mathbb{Z}$ )
1143      =  $\sum j : \text{Fin } 4, (\text{Bmat}_v * ((\text{Tmat} \wedge (m - 2))_v * \text{ones5})) j := \text{by}$ 
1144      have h := tr_validCount_even (m - 2)
1145      rw [← br_z_eq_validCount (2 * (m - 2) + 2)] at h
1146      exact h
1147      have h4 : (z (2 * (m - 4) + 2) :  $\mathbb{Z}$ )
1148      =  $\sum j : \text{Fin } 4, (\text{Bmat}_v * ((\text{Tmat} \wedge (m - 4))_v * \text{ones5})) j := \text{by}$ 
1149      have h := tr_validCount_even (m - 4)
1150      rw [← br_z_eq_validCount (2 * (m - 4) + 2)] at h
1151      exact h
1152      have hrec := asm_u_rec_sum (m - 4)
1153      rw [show (m - 4) + 3 = m - 1 from by omega, show (m - 4) + 2 = m
- 2 from by omega,
1154      show (m - 4) + 1 = m - 3 from by omega] at hrec
1155      omega
1156      rw [e4, e2, e6, e0]
1157      omega
1158      · -- 奇  $n = 2 * m + 1, 6 \leq 2 * m + 1 \implies m \geq 3$ 
1159      have hm : 3 ≤ m := by omega
1160      have e4 : 2 * m + 1 - 4 = 2 * (m - 2) + 1 := by omega
1161      have e2 : 2 * m + 1 - 2 = 2 * (m - 1) + 1 := by omega
1162      have e6 : 2 * m + 1 - 6 = 2 * (m - 3) + 1 := by omega
1163      have key : (z (2 * m + 1) :  $\mathbb{Z}$ ) + 15 * (z (2 * (m - 2) + 1) :  $\mathbb{Z}$ )
1164      = 12 * (z (2 * (m - 1) + 1) :  $\mathbb{Z}$ ) + 2 * (z (2 * (m - 3) + 1) :  $\mathbb{Z}$ )
1165      ) := by
1166      have h1 : (z (2 * m + 1) :  $\mathbb{Z}$ ) =  $\sum i : \text{Fin } 5, ((\text{Tmat} \wedge m)_v * \text{ones5}) i := \text{by}$ 
1167      have h := tr_validCount_odd m
1168      rw [← br_z_eq_validCount (2 * m + 1)] at h
1169      exact h
1170      have h2 : (z (2 * (m - 2) + 1) :  $\mathbb{Z}$ ) =  $\sum i : \text{Fin } 5, ((\text{Tmat} \wedge (m - 2))_v * \text{ones5}) i := \text{by}$ 
1171      have h := tr_validCount_odd (m - 2)
1172      rw [← br_z_eq_validCount (2 * (m - 2) + 1)] at h
1173      exact h
1174      have h3 : (z (2 * (m - 1) + 1) :  $\mathbb{Z}$ ) =  $\sum i : \text{Fin } 5, ((\text{Tmat} \wedge (m - 1))_v * \text{ones5}) i := \text{by}$ 
1175      have h := tr_validCount_odd (m - 1)
1176      rw [← br_z_eq_validCount (2 * (m - 1) + 1)] at h
1177      exact h
1178      have h4 : (z (2 * (m - 3) + 1) :  $\mathbb{Z}$ ) =  $\sum i : \text{Fin } 5, ((\text{Tmat} \wedge (m - 3))_v * \text{ones5}) i := \text{by}$ 
1179      have h := tr_validCount_odd (m - 3)
1180      rw [← br_z_eq_validCount (2 * (m - 3) + 1)] at h
1181      exact h

```

```

1181         have hrec := asm_w_rec_sum (m - 3)
1182         rw [show (m - 3) + 3 = m from by omega, show (m - 3) + 2 = m - 1
from by omega,
1183             show (m - 3) + 1 = m - 2 from by omega] at hrec
1184         omega
1185         rw [e4, e2, e6]
1186         omega
1187
1188 #print axioms c_z_rec

```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, a strict compile (0 sorry, 0 warning) of proofs/02-grid3n-colmid-indep-proved.lean exits with code 0 and empty output apart from the interface roster’s #check echoes.
- Statement integrity: the frozen statement file (61 content lines) has sha256

b726f0d8e01672b39a93ce0c256f5036014
bc80f7f0b6a5fcb484272d805c884,

the frozen interface file (8042 bytes) has sha256

66c96c3e4051ce2be5bd2c09383913f71678
e3133b0e67cff4fdc73dbed3d6e9,

and the final proof file (1188 lines) has sha256

9a8378352c7afe91ba83a519053e130c53
bd6fd7d08282c48669289682f751ac.

The audit department’s byte-exactness check reports: the first 8042 bytes of the final file equal the interface file byte for byte (this embeds the frozen statement segment verbatim, since the interface file opens with it), and the theorem statement line in the final file is byte-equal to line 48 of the frozen statement file. The declaration `c_z_rec` occurs exactly twice in the final file (the statement and the `#print axioms` command) — no second, altered statement is smuggled in.

- Kernel check: compiles with 0 `sorry` (whole-file scan including comments, also covering `admit`: zero hits); `#print axioms` output, verbatim (kernel re-run of 2026-09-30, exit code 0, kept in `audit/axioms.out`):

```
1 'c_z_rec' depends on axioms: [propext, Classical.choice, Quot.sound
  ]
```

This equals the whitelist `{propext, Quot.sound, Classical.choice}` exactly; in particular no `sorryAx` and no `Lean.ofReduceBool` (the `native_decide` axiom) appear. The single textual occurrence of `native_decide` in the final file is inside the interface’s warning docstring, which *bans* it; the axiom output is the authoritative criterion.

- Grading by the audit department: the single proven declaration is *complete proof*; the lowest grade reached for the case = *complete proof* (final adjudication `final-audit.md`: byte-exactness checks, banned-word scan, non-vacuous-lemma spot checks on the bridge and transfer lanes, axiom set check — all passed). No weakening, no added hypotheses, no unproven residue.
- Audit chain (originals in this deposit under `audit/` unless noted): the final adjudication, the axiom re-print, this note’s claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication.

5 Novelty evidence

Adjudication copied from the pipeline’s exclusivity review (C9 gate), which ran before the problem was accepted: verdict **no occupying record found** — at the object level (the counting sequence z of this specific defective strip is unregistered) and at the proposition level (no record states this recurrence for this object). Value tier **new sequence**, lower tier: the “new sequence” half holds (z itself is unregistered in OEIS across five query forms, and the defective-strip object is unnamed in the literature layer), while the “new recurrence” half does *not* hold and is not claimed: the coefficient signature $(12, -15, 2)$ of the parity subsequences’ order-3 recurrences is already indexed in the OEIS linear-recurrence index, and the even-indexed subsequence $z(2), z(4), \dots$ coincides term-by-term (over their overlap window) with the registered sequence A122011 [3] — a boundary that must be stated

whenever that subsequence is mentioned. A122011’s own record is a 3×3 matrix-power exercise with no graph-theoretic or independent-set content, so the object is different even where the values agree; the odd-indexed subsequence has no registered match. The nearest literature neighbours are delimited: the Merrifield–Simmons line [5] treats the quantity $\sigma(G - S)$ for vertex-deleted graphs as a tool in sign-relational results (not this counting sequence), the De Ita et al. branching work [6] counts independent sets of *intact* grid graphs by vertex elimination (no deleted-column family), and the independence-complex work [7] concerns square-grid complexes (no counting sequence). Summary by channel; the full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md` (verbatim, byte-for-byte).

channel	queries	result
OES, numeric	4 prefix-window strings (main; drop-first-1; drop-first-2; odd-indexed subsequence)	all zero hits; positive-control anchor hit (A051756, the intact-strip sequence)
OES, numeric (even subsequence)	1 string	hit A122011: same values, different object (matrix-power example); recurrence signature already indexed there
OES, keyword	6 keyword strings (defective/periodic grid, independent sets, deleted vertices)	no this-object hit
arXiv API	11 strings (object-name $\times$ property words, plus variants)	all returned; Top-3 candidates read at full-text level, none this object
Crossref	4 strings	one high-relevance record (De Ita et al. 2024, intact grids); not this object
Mathematics Stack Exchange	2 site-specific strings	no this-object hit
local formalisation ecosystem	6 GitHub strings + 4 mathlib rg sweeps	counting object absent; no same-form formalisation
residual exposure	—	the literature-layer zero-hit verdict is limited to arXiv + Crossref + MSE; OpenAlex / zbMATH / general web search were not reachable

The channels are named so a later reader can re-run them; if any channel later returns a same-form record, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record.

6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The kernel layer proves Theorem 1 and nothing else. (b) The grading is *complete proof* for that theorem only; this note makes no claim about recurrence minimality (that no shorter-order recurrence holds is neither proved nor searched), about closed forms, or about asymptotics. (c) The identification of **z** with the independent-set count of the defective strip is definitional in the formalisation (**z** is defined as that count), and the bridge lemma is kernel-proved; no further combinatorial conjecture layer is carried. (d) The novelty verdict’s literature-layer half is limited to three sources (arXiv, Crossref, MSE); OpenAlex, zbMATH and general web search were not reachable in the review. (e) The initial values stated in the frozen statement file were verified there by `native_decide`, which is a data check outside the axiom whitelist and deliberately excluded from the deposited proof; the deposited proof derives everything it needs through the transfer-matrix route. (f) The gate-four sign-off of the underlying task was exercised by the author’s instruction of 2026-09-30 to run both the paper lane and this claim lane with GitHub out-bound authorisation, while the task report file header retains its pre-sign-off

wording unchanged; both calibers are recorded. (g) This note carries no literature review and is a deposit record, not a survey; the narrative paper is prepared separately under the pipeline’s paper lane and will reference this deposit.

Data availability

The deposit package for this claim contains the claim note (LaTeX source and PDF), the Lean sources (`proofs/`: frozen statement file, frozen interface file, final proof file, `lean-toolchain`), the verification and audit logs, and the full C9 query log with zero-hit entries (`audit/`). Public mirror: the claim lane’s repository <https://github.com/Aurora0134/ai4math-claims>, directory `grid3n-colmid-indep/` (pushed under the author’s instruction of 2026-09-30; the commit and timestamp are recorded in the source repository’s `claims/grid3n-colmid-indep/card.md`). Zenodo: the DOI is reserved at upload by the author and is recorded in `metadata/README.md` of this package once created; see `UPLOAD.md` in the source repository. Claim note under CC BY 4.0, code under MIT.

Use of generative AI

(a) AI performed: candidate generation and the exclusivity review, auto-formalisation of the definitions and statement, the numerical cross-check of the proof route (three independent counting implementations over $n = 0..20$), proof search and proof-script writing (eight worker agents in a workflow: blueprint, blueprint review, bridge, transfer, assembly, final audit, report, reader audit), and the assembly of this note including its verbatim listings. (b) Model, node and budget: the proving phase ran on the session node of the pipeline host (the wire designation `a6api-main/glm-5.3-flash` is recorded in the claim-lane card’s node snapshot, `claims/grid3n-colmid-indep/card.md`; the source task ledger records the same session’s node as its session node), with no cross-node switching, consuming 8 agent-run samples of a permitted 16, 0 llm-calls of a permitted 8, and 10 strict kernel compiles of a permitted 36 (plus compile-debug probes, separately ledgered), copied verbatim from the task ledger. This note consumed 0 pipeline LLM calls against a cap of 4; drafting was mechanical assembly. (c) Final adjudication: all statements and proofs reported here were checked by the Lean 4 kernel: the proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound,`

Classical.choice}. (d) The AI system is not an author; the human author reviewed the evidence and is responsible for all content.

Author contributions

CRediT-style: the human author adjudicated topic selection, statement semantics and the deposit decision; the AI4Math pipeline (an LLM) generated the candidate propositions, the proof searches and the text drafts. The AI system is not listed as an author.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] OEIS Foundation. Sequence A122011 (generating function $x^2(1+x)/(1-12x+15x^2-2x^3)$; the 3×3 matrix-power example of A. M. Robert, *Linear Algebra, Examples and Applications*, World Scientific, 2005, p. 58; linear-recurrence index, signature (12, −15, 2)). *The On-Line Encyclopedia of Integer Sequences*, entry A122011. <https://oeis.org/A122011>
- [4] OEIS Foundation. Sequence A051736 (number of independent sets of the $3 \times n$ grid strip; positive-control anchor of the review). *The On-Line Encyclopedia of Integer Sequences*, entry A051736. <https://oeis.org/A051736>
- [5] M. Trinks. The Merrifield–Simmons conjecture holds for bipartite graphs. *arXiv:1006.4253*, 2021 (v2). <https://arxiv.org/abs/1006.4253>
- [6] G. De Ita, M. Bello-López, and J. Marcial-Romero. Counting rules for computing the number of independent sets of a grid graph. *Mathematics* 12(6):922, 2024. <https://doi.org/10.3390/math12060922>
- [7] J. Jonsson. On the independence complex of square grids. *arXiv:math/0701890*, 2007 (v2). <https://arxiv.org/abs/math/0701890>